

MecaMind 从零搭建完整教程

本教程手把手复盘整个 MecaMind 项目是如何一步步搭建起来的：从装环境、建工作区，到机器人建模、双仿真后端、SLAM 建图、Nav2 导航、任务层和自动验收。每一步都包含 **目标 → 原理 → 操作 → 验证 → 常见错误** 五个环节。面向 ROS 2 初学者，假设你使用 Windows 10 + WSL2 (Ubuntu 24.04)。

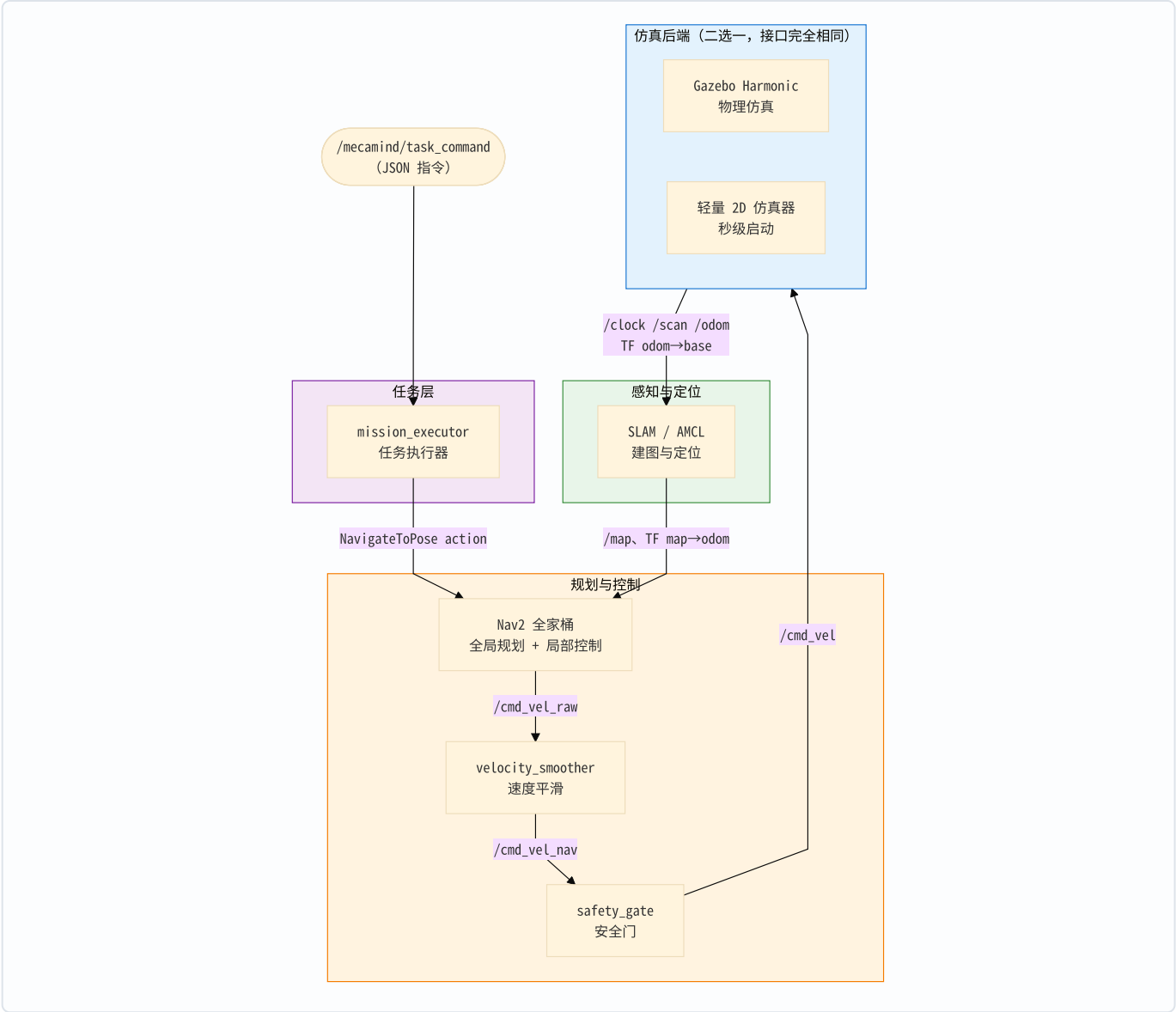
教程里引用的所有代码都真实存在于本仓库对应路径中，你可以边读边对照源码。每节课的成品可以用 `git` 标签直接取到（`lesson1.0`、`lesson2.0`），**但强烈建议照着教程亲手敲一遍**，遇到问题再对照成品。

目录

- 第 0 步：准备开发环境
- 第 1 步：创建工作区与包骨架
- 第 2 步：用 URDF 描述机器人
- 第 3 步：Gazebo 世界与机器人物理模型
- 第 4 步：用 `ros_gz_bridge` 打通 Gazebo 与 ROS 2
- 第 5 步：自研轻量 2D 仿真器
- 第 6 步：统一接口——双后端可互换的关键
- 第 7 步：安全门——速度链路路上的保险丝
- 第 8 步：接入 SLAM Toolbox 建图
- 第 9 步：半自动建图——路线驱动器
- 第 10 步：地图质量分析与资产管理
- 第 11 步：CP2 自动验收脚本
- 第 12 步：搭建 Nav2 导航栈
- 第 13 步：任务层——命名目标与任务执行器
- 第 14 步：CP3 验收与里程计漂移观察
- 第 15 步：调试方法与避坑手册
- 附录 A：命令速查表
- 附录 B：项目文件地图
- 附录 C：里程碑自查表
- 附录 D：如何把本项目改造成你自己的机器人
- 附录 E：术语表

全局架构：先看懂我们要搭什么

在动手之前，先把最终形态放在眼前。整个项目搭完后的数据流是这样的：



搭建顺序就是本教程的章节顺序，遵循一条原则：**先让数据流的上游跑通，再接下游**。仿真器是一切的源头（第 3~5 步），有了传感器数据才能做 SLAM（第 8 步），有了地图才能做导航（第 12 步），最后才是任务层（第 13 步）。

教程步骤与课程、git 标签的对应关系：

教程步骤	对应课程	git 标签	里程碑
第 0~4 步	第一课 10.3.1 项目总览与仿真环境	lesson1.0	Gazebo 里能遥控开车（CP1）
第 5~11 步	第二课 10.3.2 SLAM 建图与地图管理	lesson2.0	自动建出合格地图（CP2）
第 12~14 步	第三课 10.3.3 Nav2 导航与路径规划	—	JSON 指令自主导航（CP3）
第 15 步 + 附录	贯穿全程	—	独立排障能力

第 0 步：准备开发环境

装环境就像装修房子先通水电：这一步做的事枯燥、成果也看不见，但后面每一步都踩在它上面。水电走错了，铺完地板才发现要砸开重来——环境里一个 `DISPLAY` 设错，往往到第 8 步启动 RViz 时才暴露。所以请耐心把本步的每个验证项都跑通再往下走。

0.1 目标

在 Windows 10 上得到一个可运行 GUI 程序的 Ubuntu 24.04 环境，装好 ROS 2 Jazzy 与 Gazebo Harmonic。

0.2 原理：为什么是 WSL2 + Jazzy + Harmonic

- **WSL2** 是 Windows 内置的 Linux 虚拟化方案，自带 WSLg (GUI 支持)，不用装双系统或 VirtualBox 就能跑 RViz、Gazebo 的图形界面。
- **ROS 2 Jazzy** (2024.5 发布) 是面向 Ubuntu 24.04 的 LTS 版本，支持到 2029 年。
- **Gazebo Harmonic** 是与 Jazzy 官方配对的仿真器版本（注意：新版 Gazebo 命令是 `gz sim`，和老的 Gazebo Classic `gazebo` 命令完全是两套软件）。

0.3 操作

① 安装 WSL2 + Ubuntu 24.04 (在 Windows PowerShell 管理员模式下)：

```
wsl --install -d Ubuntu-24.04
# 装完重启，首次进入会让你设置 Linux 用户名和密码
```

② 安装 ROS 2 Jazzy (以下全部在 WSL 的 bash 里执行)：

```
# 设置 locale
sudo apt update && sudo apt install -y locales
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8

# 添加 ROS 2 软件源
sudo apt install -y software-properties-common curl
sudo add-apt-repository universe
sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key \
-o /usr/share/keyrings/ros-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/ros-archive-keyring.gpg] \
http://packages.ros.org/ros2/ubuntu $(. /etc/os-release && echo $UBUNTU_CODENAME) main" \
| sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null

# 安装桌面完整版 (含 RViz) + 开发工具
sudo apt update
sudo apt install -y ros-jazzy-desktop ros-dev-tools
```

③ 安装 Gazebo Harmonic 与本项目用到的 ROS 包：

```
sudo apt install -y \
  ros-jazzy-ros-gz \
  ros-jazzy-slam-toolbox \
  ros-jazzy-navigation2 \
  ros-jazzy-nav2-bringup \
  ros-jazzy-xacro \
  ros-jazzy-tf2-tools \
  python3-pytest
```

④ 配置 shell 环境（写进 `~/.bashrc`，每个新终端自动生效）：

```
cat >> ~/.bashrc <<'EOF'
# ROS 2 环境
source /opt/ros/jazzy/setup.bash
# WSLg 的 X server 固定在 :0，避免 DISPLAY 被误设成 Windows IP
export DISPLAY=:0
# 用固定的 DOMAIN_ID 隔离自己的 ROS 网络（0~232 任选，同网段同 ID 才互通）
export ROS_DOMAIN_ID=42
EOF
source ~/.bashrc
```

0.4 验证

```
# ① ROS 2 正常：应打印 jazzy
echo $ROS_DISTRO

# ② GUI 正常：应弹出两个小海龟窗口（Ctrl+C 退出）
ros2 run turtlesim turtlesim_node

# ③ Gazebo 正常：应弹出 Gazebo 窗口（关闭窗口退出）
gz sim shapes.sdf
```

0.5 多终端工作流（贯穿全教程的操作习惯）

ROS 2 开发的日常形态是**同时开 3~5 个终端**：一个跑 launch、一个遥控、一个看话题、一个随时敲诊断命令。Windows 10 推荐装 **Windows Terminal**（微软商店免费），用法：

- `Ctrl+Shift+T` 新开标签页，下拉菜单选 Ubuntu-24.04 直接进 WSL；
- 或者 `Alt+Shift+D` 纵向分屏，左右两个终端并排盯；
- **每个新终端都要有环境**：如果你照 0.3 ④ 把 `source` 写进了 `~/.bashrc`，新终端自动就绪；工作区的 `install/setup.bash` 则需要在每个用到本项目包的终端里手动 `source`（或也追加进 `bashrc`）。

本教程后文写“另开终端执行 xxx”时，指的就是新开一个这样的标签页。

0.6 常见错误

现象	原因与解法
qt.qpa.xcb: could not connect to display 172.x.x.x:0	DISPLAY 被设成了 Windows 主机 IP。执行 <code>export DISPLAY=:0</code>
gz: command not found	没装 <code>ros-jazzy-ros-gz</code> ，或者新终端没 <code>source /opt/ros/jazzy/setup.bash</code>
两台电脑上互相看到对方的话题	<code>ROS_DOMAIN_ID</code> 相同。各自设不同的值
新终端里 <code>ros2 run</code> 找不到 <code>mecamind</code> 包	忘了 <code>source ~/mecamind_ros2/install/setup.bash</code>

第 1 步：创建工作区与包骨架

可以把工作区想象成一个工厂：`src/` 是原料区（你写的源码）、`build/` 是加工车间（编译中间产物）、`install/` 是成品仓库（真正被运行的东西）。`colcon build` 就是开动流水线，而 `source install/setup.bash` 相当于告诉系统"以后找工具去成品 仓库拿"。理解了这条流水线，"改了代码为什么不生效"这类新手 问题就有了答案——原料改了，成品还没重新生产。

1.1 目标

建立标准的 `colcon` 工作区，创建本项目的四个包，理解"包"的组织方式。

1.2 原理：工作区、包、colcon 是什么关系

- **工作区 (workspace)**：一个包含 `src/` 目录的文件夹。`colcon build` 会把 `src/` 下所有包编译到 `build/`，安装到 `install/`。
- **包 (package)**：ROS 2 代码的发布单元，每个包有一个 `package.xml` 声明名字和依赖。Python 包再加一个 `setup.py` 说明怎么安装。
- **职责拆分**：我们按"描述 / 仿真 / 工具 / 启动"把代码拆成四个包，这是 ROS 社区的惯例——描述文件（URDF）和算法代码分开，方便复用：

包名	职责
<code>mecamind_description</code>	机器人的 URDF 模型（给 ROS/Rviz 用）
<code>mecamind_gazebo</code>	Gazebo 的模型、世界、桥接配置
<code>mecamind_tools</code>	所有 Python 节点：轻量仿真器、SLAM 工具、任务层…
<code>mecamind_bringup</code>	顶层 launch 入口（日常使用只需要记住这个包）

1.3 操作

```
# ① 创建工作区
mkdir -p ~/mecamind_ros2/src
cd ~/mecamind_ros2

# ② 创建四个包（ament_python 类型；description/gazebo 只放资源文件，
# 用 ament_cmake 也行，本项目统一用简单的结构）
cd src
ros2 pkg create mecamind_description --build-type ament_python
ros2 pkg create mecamind_gazebo      --build-type ament_python
ros2 pkg create mecamind_tools       --build-type ament_python --dependencies rclpy
ros2 pkg create mecamind_bringup     --build-type ament_python
cd ..

# ③ 第一次构建（此时都是空包，验证工具链）
colcon build
source install/setup.bash
```

重要习惯：以后每次改了代码（包括 launch、config），都要 `colcon build --packages-select <包名>` 再 `source install/setup.bash`。Python 包默认不是符号链接安装，源码改了不重新 build 是不生效的（开发期可以用 `colcon build --symlink-install` 省去重复构建）。

1.4 数据文件怎么装进包里：setup.py 的 data_files

Python 包默认只安装 *.py。launch 文件、YAML 配置、地图、RViz 布局 都要在 setup.py 的 data_files 里显式声明，例如本项目的 `src/mecamind_tools/setup.py`：

```
data_files=[
    ("share/ament_index/resource_index/packages", [f"resource/{package_name}"]),
    (f"share/{package_name}", ["package.xml"]),
    (os.path.join("share", package_name, "launch"), glob("launch/*.launch.py")),
    (os.path.join("share", package_name, "config"), public_config_files),
    (os.path.join("share", package_name, "maps"), glob("maps/*")),
    (os.path.join("share", package_name, "rviz"), glob("rviz/*.rviz")),
    (os.path.join("share", package_name, "worlds"), glob("worlds/*.world")),
],
```

运行时用 `get_package_share_directory("mecamind_tools")` 就能定位到 `install/mecamind_tools/share/mecamind_tools/`，从那里读配置。

可执行节点则登记在 `entry_points`，登记后才能 `ros2 run`：

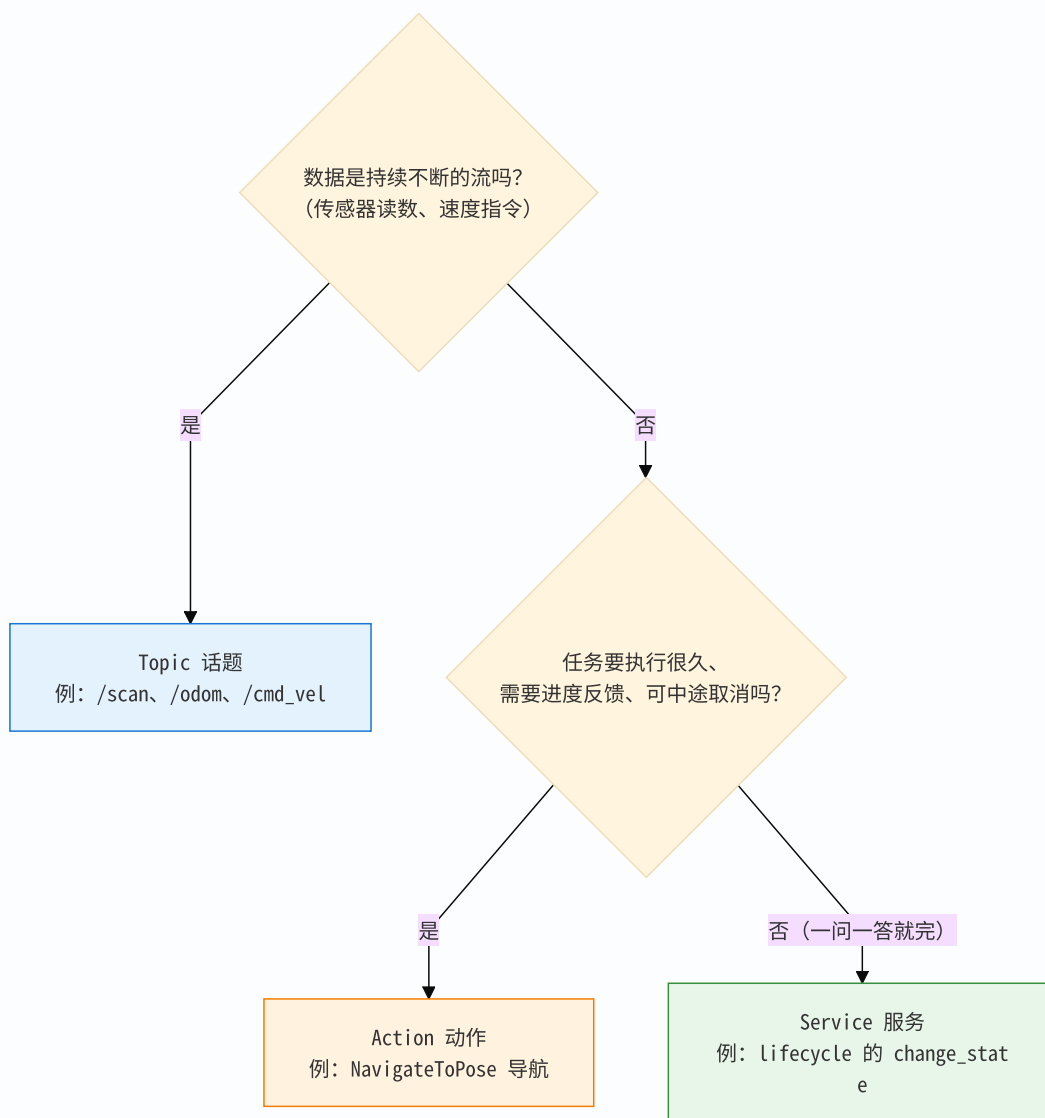
```
entry_points={
    "console_scripts": [
        # "命令名 = 模块路径:函数"
        "mecamind_simulator_node = mecamind_tools.simulator_node:main",
    ],
},
```

1.5 ROS 2 核心概念

后面每一步都会反复用到这六个概念，先在这里一次讲清（已经熟悉 可跳过本节）：

- ① **节点 (Node)** —— 一个独立运行的功能单元（进程）。本项目最终 会同时跑约 15 个节点：仿真器是节点、SLAM 是节点、安全门也是节点。 `ros2 node list` 随时可以看当前活着的节点。
- ② **话题 (Topic)** —— 节点间的"广播频道"。发布者往频道里扔消息， 订阅者收。特点是**多对多、异步、发后不管**。传感器数据流（`/scan`、`/odom`）都用话题。
- ③ **服务 (Service)** —— "一问一答"的远程调用。例如 lifecycle 的 `change_state` 就是服务：激活器请求、目标节点答复成功与否。
- ④ **动作 (Action)** —— "有受理、有进度反馈、可取消"的长任务协议。 导航去某个点要花几十秒，用话题（发后不管）和服务（阻塞等答复） 都不合适，action 专为这种场景设计。第 13 步会深入。
- ⑤ **参数 (Parameter)** —— 节点的运行时配置。可以在 launch 里传、 命令行传（`--ros-args -p name:=value`）、YAML 文件批量加载。 `use_sim_time` 就是每个节点都有的内置参数。
- ⑥ **TF (坐标变换)** —— 回答"A 坐标系下的点在 B 坐标系下是多少" 的分布式变换库。各节点把自己知道的变换段广播到 `/tf`，任何节点 都能拼出完整的变换链做查询。**移动机器人开发一半的 bug 出在 TF**， 所以本教程会反复强调 TF 树的形态。

一图总结通信方式的选择：



1.6 验证

```
# 四个包都能被找到即成功
ros2 pkg list | grep mecanimind
```

第 2 步：用 URDF 描述机器人

URDF 好比机器人的"骨架 X 光片"：它只回答"哪块骨头（连杆） 通过哪个关节接在哪里"，不关心体重、肌肉力量这些物理属性—— 那些是第 3 步 SDF 的事。X 光片虽然简单，却是一切的基准：RViz 靠它画模型，TF 靠它算"雷达在底盘的哪个位置"。骨架画错 一厘米，后面所有激光点就都偏一厘米。

2.1 目标

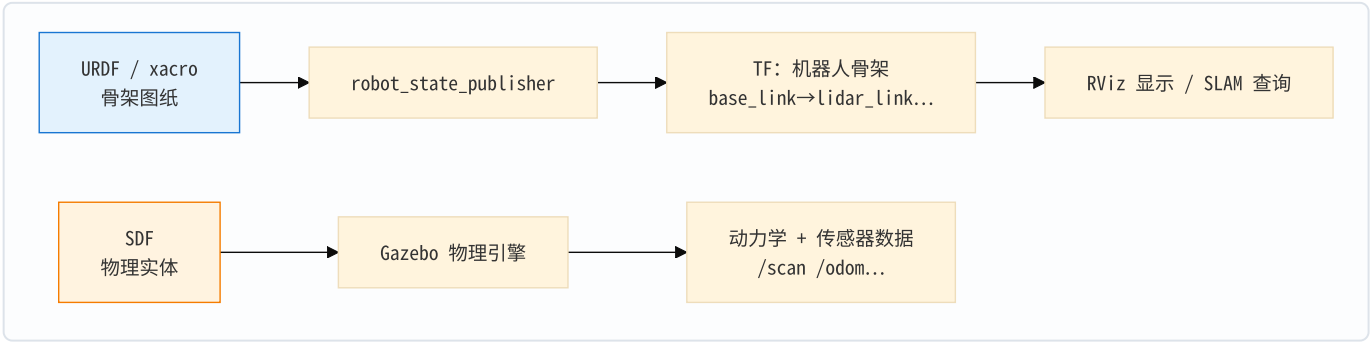
写出机器人的"骨架图" `mecanimind_mecanum.urdf.xacro`，让 `robot_state_publisher` 能发布 TF、RViz 能显示模型。

2.2 原理：URDF 与 SDF 的分工（初学者最容易混淆的点）

机器人在本项目里有两份描述文件，各司其职：

	URDF（本步）	SDF（第 3 步）
给谁用	ROS: robot_state_publisher、RViz	Gazebo 物理引擎
描述什么	连杆装配关系（TF 树）、外观	质量、摩擦、传感器、驱动插件
文件	mecamind_description/urdf/mecamind_mecanum.urdf.xacro	mecamind_gazebo/models/mecamind_mecanum/model.sdf

两份文件的几何尺寸必须一致（轮距、雷达安装位置），否则 TF 和物理对不上，激光点会"长"在错误的位置上。



另一个关键概念是 **base_footprint**：一个没有实体的虚拟连杆，表示机器人在地面上的投影点。ROS 导航约定用它当机器人根坐标系（始终贴地 z=0），SLAM 和 Nav2 的 `base_frame` 参数都填它。

2.3 操作：编写 xacro

xacro 是 XML 宏语言，`property` 定义常量、`macro` 定义模板。四个轮子只差安装位置，用一个宏生成，避免复制粘贴四遍。完整文件见 `src/mecamind_description/urdf/mecamind_mecanum.urdf.xacro`，骨干结构如下：

```

<robot xmlns:xacro="http://www.ros.org/wiki/xacro" name="mecamind_mecanum">
  <!-- 几何常量: 轮半径 7.5cm, 轮心前后 ±0.20m、左右 ±0.17m -->
  <xacro:property name="wheel_radius" value="0.075"/>
  <xacro:property name="wheel_x" value="0.20"/>
  <xacro:property name="wheel_y" value="0.17"/>

  <!-- 根坐标系: 贴地虚拟连杆 -->
  <link name="base_footprint"/>

  <!-- 底盘: 0.42 x 0.32 x 0.12 m 的盒子 -->
  <link name="base_link"> ... </link>
  <joint name="base_footprint_joint" type="fixed">
    <parent link="base_footprint"/> <child link="base_link"/>
  </joint>

  <!-- 轮子宏: 一个模板生成四个轮子 -->
  <xacro:macro name="wheel" params="name x y">
    <link name="${name}_wheel"> ... </link>
    <joint name="${name}_wheel_joint" type="continuous">
      <origin xyz="{x} {y} {wheel_radius}"/>
      <axis xyz="0 1 0"/> <!-- 转轴沿 y: 普通轮子的滚动轴 -->
    </joint>
  </xacro:macro>

  <xacro:wheel name="front_left" x="{wheel_x}" y="{wheel_y}"/>
  <xacro:wheel name="front_right" x="{wheel_x}" y="{wheel_y}"/>
  <xacro:wheel name="rear_left" x="{wheel_x}" y="{wheel_y}"/>
  <xacro:wheel name="rear_right" x="{wheel_x}" y="{wheel_y}"/>

  <!-- 传感器连杆: 雷达 (前部偏上)、IMU、相机 -->
  <link name="lidar_link"> ... </link>
  <joint name="lidar_joint" type="fixed">
    <origin xyz="0.08 0 0.19"/>
    <parent link="base_link"/> <child link="lidar_link"/>
  </joint>
</robot>

```

三个必须记住的细节:

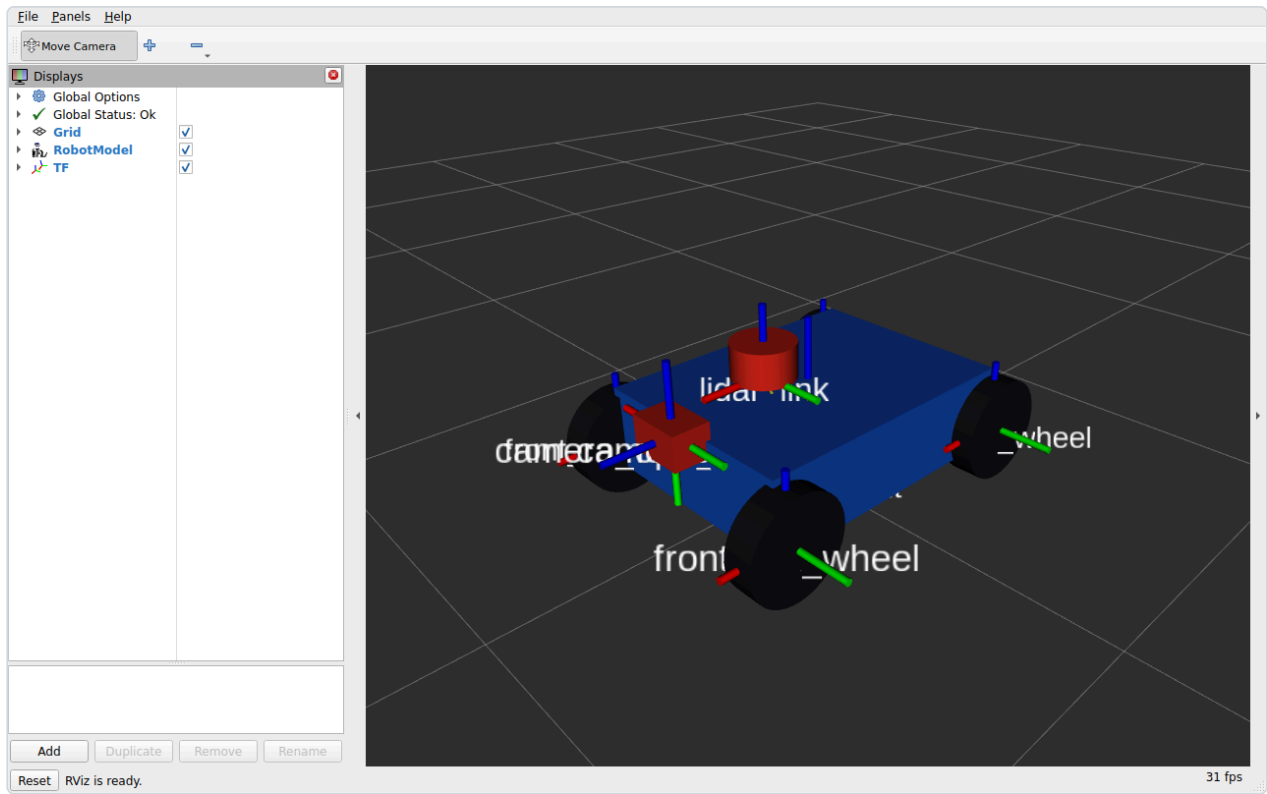
1. **lidar_link** 这个名字很重要——它必须与激光消息的 `header.frame_id` 一致 (第 3 步在 Gazebo 传感器里配置 `gz_frame_id`), SLAM 才能把激光点变换到底盘坐标系。
2. **fixed** 关节的 TF 是常量, `robot_state_publisher` 作为静态变换 一次性发布; **continuous** 是可无限旋转的轮关节。
3. 相机还需要一个 **camera_optical_frame**: ROS 图像处理约定 z 轴朝前、x 右、y 下, 与机器人本体 (x 前、y 左、z 上) 不同, 要绕两个轴各转 -90°。

2.4 验证

```
# ① xacro 能展开成合法 URDF
xacro src/mecamind_description/urdf/mecamind_mecanum.urdf.xacro > /tmp/robot.urdf
check_urdf /tmp/robot.urdf      # 应打印连杆树，root 是 base_footprint

# ② 在 RViz 里看模型 (urdf_launch 提供通用显示入口，
#   joint_state_publisher_gui 提供轮子滑条)
sudo apt install -y ros-jazzy-urdf-launch ros-jazzy-joint-state-publisher-gui
ros2 launch urdf_launch display.launch.py \
  urdf_package:=mecamind_description \
  urdf_package_path:=urdf/mecamind_mecanum.urdf.xacro
```

在 RViz 里看到的效果如下：蓝色方块是底盘（base_link），四个黑色 圆柱是麦轮，顶部红色圆柱是雷达（lidar_link），车头红色方块是 相机。每个连杆上的红绿蓝三色小坐标轴就是 TF 帧——URDF 写的 `<origin>` 数值直接决定了它们的位置：



RViz 中显示的 URDF 机器人模型，可以看到各连杆及其 TF 坐标轴

2.5 常见错误

现象	原因与解法
RViz 里模型全白且报 <code>No transform from [base_link]</code>	Fixed Frame 没选对，改成 <code>base_footprint</code>
<code>check_urdf</code> 报 root 不唯一	有连杆没用 <code>joint</code> 接到树上，URDF 必须是一棵树
改了 xacro 但 RViz 没变化	忘了重新 <code>colcon build</code> （或未用 <code>--symlink-install</code> ）

2.6 动手练习

1. 把雷达安装位置从 `xyz="0.08 0 0.19"` 改到车尾，重新在 RViz 里查看——理解 URDF 改动如何直接反映到 TF 和显示；
2. 用 `ros2 run tf2_ros tf2_echo base_footprint lidar_link` 打印 雷达相对底盘的变换，对照你在 URDF 里写的数值；
3. 思考题：为什么 `base_footprint` 要贴地 ($z=0$) 而不是放在 底盘中心？提示：想想 2D 地图上"机器人的位置"指的是哪个点。

第 3 步：Gazebo 世界与机器人物理模型

本步最有意思的知识点是麦克纳姆轮的仿真。先想想冰刀：沿刀刃 方向几乎无摩擦、可以自由滑行，垂直方向却牢牢咬住冰面——同一个接触点，摩擦力随方向而不同，这叫**各向异性摩擦**。麦轮上 45° 斜置的小辊子本质上就是给每个轮子装了一副"斜着放的冰刀"：沿辊子轴向自由滑、垂直方向正常抓地。抓住这一点，你就能看懂 为什么仿真里不用建几十个小辊子，改两个摩擦系数就够了。

3.1 目标

搭出"三室户"仿真环境，写出带麦轮物理、激光雷达、IMU、相机和 驱动插件的机器人 SDF 模型，机器人能在 Gazebo 里被 `/cmd_vel` 驱动。

3.2 原理 ①：三室户世界怎么设计

世界文件 `src/mecamind_gazebo/worlds/three_room_house.sdf`：7.8m x 5.9m 的外墙围出一套房，两道隔断墙分出客厅（左上）、卧室（右上）、下方门厅走廊，再摆桌子、柜子、方柱三件家具。

设计考虑：

- 墙高 0.8m、厚 0.12m：雷达装在 0.19m 高，扫墙很稳；
- 所有墙和家具 `static=true`：不参与动力学，撞不动，物理计算省；
- 有门洞、有回环路线：房间之间能绕圈走，给 SLAM 回环检测 制造机会（第 8 步会用到）；
- 世界里 `<include><uri>model://mecamind_mecanum</uri></include>` 直接引入机器人，出生点写在模型自己的 `<pose>` 里。

3.3 原理 ②：麦克纳姆轮怎么在 Gazebo 里"仿真"

这是全项目最巧妙的一段。真实麦轮靠 45° 斜置的小辊子实现全向移动，但仿真里不需要建出几十个小辊子，用**各向异性摩擦**数学等价：

```
<!-- src/mecamind_gazebo/models/mecamind_mecanum/model.sdf 轮子碰撞体 -->
<collision name="collision">
  <geometry><sphere><radius>0.075</radius></sphere></geometry>
  <surface><friction><ode>
    <mu>1.0</mu>      <!-- 第一摩擦方向：正常抓地 -->
    <mu2>0.0</mu2>    <!-- 第二摩擦方向：完全打滑 = 小轮子自由滚 -->
    <fdirl gz:expressed_in="base_link">1 -1 0</fdirl> <!-- 摩擦主方向 45° 斜置 -->
  </ode></friction></surface>
</collision>
```

- 碰撞体用球而不是圆柱：与地面单点接触，物理求解更稳定；
- `fdirl` 左前/右后轮是 `(1,-1,0)`，右前/左后轮是 `(1,1,0)` —— 对角线成对的摩擦方向布置正是麦轮全向移动的原理；
- 四个轮速的不同组合就能合成前进、横移、斜移、原地转。

驱动交给 Gazebo 自带的 MecanumDrive 插件：

```
<plugin filename="gz-sim-mecanum-drive-system" name="gz::sim::systems::MecanumDrive">
  <wheel_separation>0.34</wheel_separation> <!-- 必须与 URDF 轮距一致 -->
  <wheelbase>0.40</wheelbase>
  <wheel_radius>0.075</wheel_radius>
  <topic>/cmd_vel</topic>      <!-- 订阅速度指令 -->
  <odom_topic>/odom</odom_topic> <!-- 发布轮速积分的里程计 -->
  <tf_topic>/gz/tf</tf_topic>   <!-- 发布 TF odom->base_footprint -->
  <frame_id>odom</frame_id>
  <child_frame_id>base_footprint</child_frame_id>
</plugin>
```

重点理解：这份里程计来自轮速积分，而麦轮横移时轮子必然打滑，所以 `/odom` 存在真实的累积漂移——这正是第 14 步漂移矫正专题的素材。

3.4 原理 ③：传感器

在 `lidar_link` 上挂 `gpu_lidar` 传感器：360 束、±180°、0.12~8m、10Hz，外加 0.005m 高斯噪声模拟真实雷达。关键行：

```
<sensor name="lidar" type="gpu_lidar">
  <topic>/scan</topic>
  <gz_frame_id>lidar_link</gz_frame_id> <!-- 必须与 URDF frame 同名! -->
  ...
</sensor>
```

3.5 操作

1. 建目录 `src/mecamind_gazebo/models/mecamind_mecanum/`，写 `model.sdf` 和 `model.config`（后者是模型清单，几行 XML）；
2. 写世界文件 `src/mecamind_gazebo/worlds/three_room_house.sdf`；
3. 在 launch 里把 `models/` 目录加入 Gazebo 搜索路径（第 4 步统一做）：

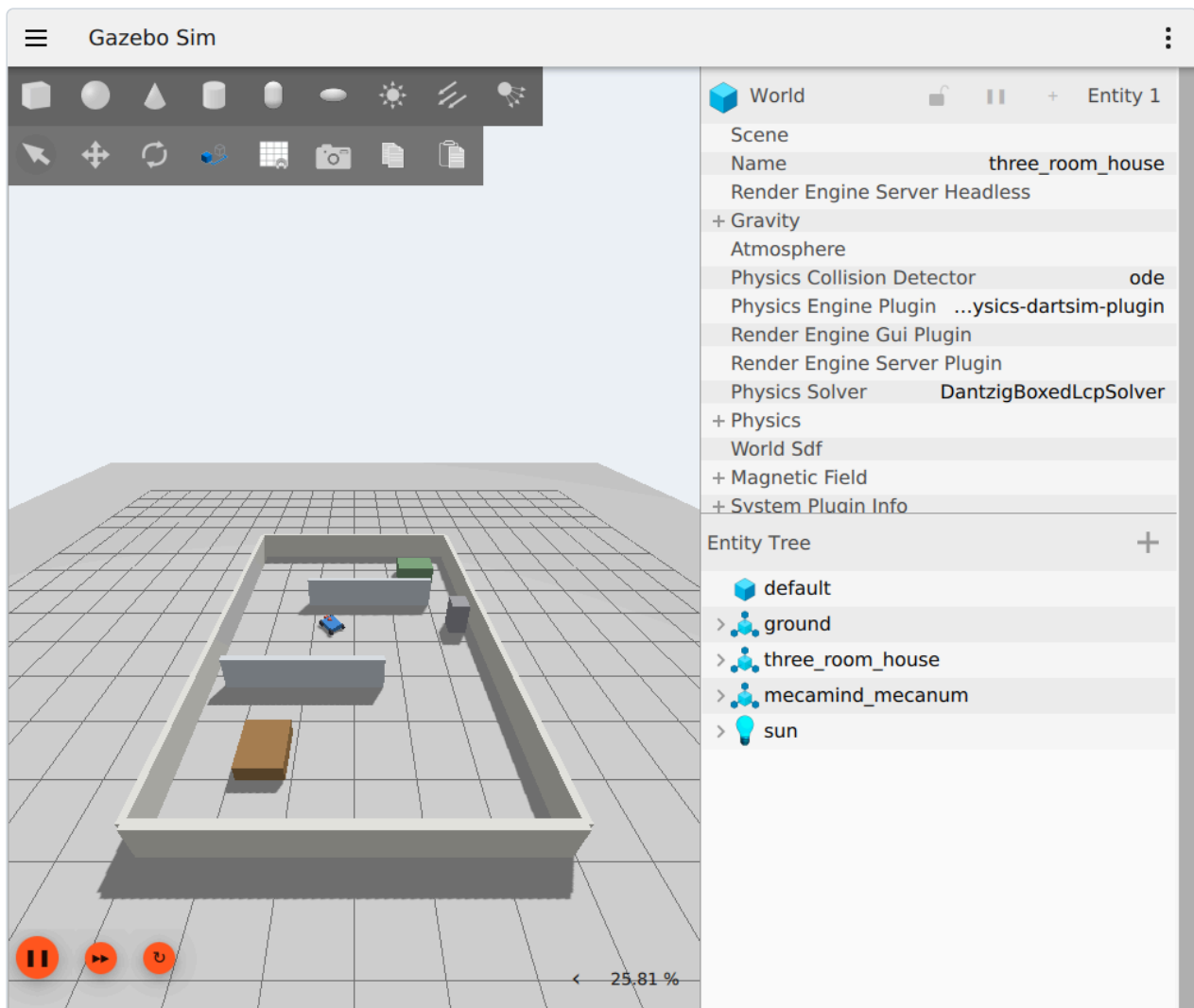
```
SetEnvironmentVariable(
    "GZ_SIM_RESOURCE_PATH",
    [model_path, ":", EnvironmentVariable("GZ_SIM_RESOURCE_PATH", default_value="")],
)
```

3.6 验证（纯 Gazebo，不经过 ROS）

```
GZ_SIM_RESOURCE_PATH=~/.mecamind_ros2/src/mecamind_gazebo/models \
gz sim -r src/mecamind_gazebo/worlds/three_room_house.sdf

# 另开一个终端，直接用 gz 命令发速度（注意这是 Gazebo 话题，不是 ROS 话题）
gz topic -t /cmd_vel -m gz.msgs.Twist -p 'linear: {x: 0.2}'
# 看激光原始数据
gz topic -e -t /scan | head -30
```

机器人能动、激光有数据，说明纯 Gazebo 侧已经完整。启动后的画面应该是这样——三房间户型、两道隔断墙、桌子/柜子/方柱三件家具，蓝色小车停在屋内；右侧 Entity Tree 里能看到 `three_room_house` 和 `mecamind_mecanum` 两个模型都已加载：



Gazebo 中的三房间世界与麦轮机器人

3.7 动手练习

1. 发 `gz topic -t /cmd_vel -m gz.msgs.Twist -p 'linear: {y: 0.2}'` (纯横移)，观察麦轮车"平移不转头"——差速车做不到这一点；
2. 把某个轮子的 `fdir1` 方向故意改错 (比如四个全改成 `1 1 0`)，重启仿真再发横移指令，观察车"想横移却在打转"——亲身体会 对角线摩擦方向布置的必要性 (改完记得改回来)；
3. 在世界文件里加一个新障碍 (照抄 `pillar` 改坐标)，确认激光能扫到它。

第 4 步：用 `ros_gz_bridge` 打通 Gazebo 与 ROS 2

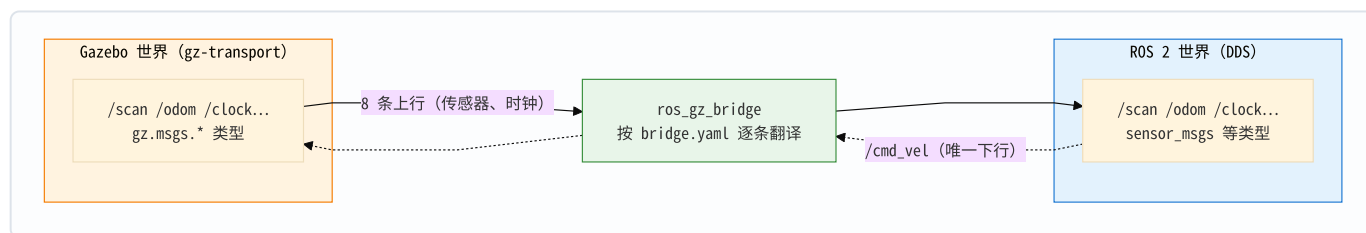
Gazebo 和 ROS 2 像两个语言不通的国家：各说各的话 (`gz-transport` vs `DDS`)，连"激光数据"这个词的写法都不同 (`gz.msgs.LaserScan` vs `sensor_msgs/msg/LaserScan`)。`ros_gz_bridge` 就是边境上的 翻译官，手里拿着一张报关清单 (`bridge.yaml`)，清单上登记了 哪些货物 (话题) 允许通行、往哪个方向走、两边各叫什么名字——没登记的，一律不通。

4.1 目标

让 ROS 2 节点能订阅 Gazebo 的传感器、能给 Gazebo 发速度指令，一条 `ros2 launch` 命令拉起完整的 Gazebo 后端。

4.2 原理：为什么需要"桥"

Gazebo 与 ROS 2 是**两套独立的通信系统**：Gazebo 用 `gz-transport`，ROS 2 用 `DDS`，消息类型也不同 (`gz.msgs.LaserScan` vs `sensor_msgs/msg/LaserScan`)，互相看不见对方的话题。`ros_gz_bridge` 的 `parameter_bridge` 按映射表逐条建立"翻译通道"。



映射表 `src/mecamind_gazebo/config/bridge.yaml` 一共桥 9 条话题，每条五要素 (两侧话题名、两侧类型、方向)。方向的选择很讲究——**单向桥省资源、也避免消息回环**：

ROS 话题	方向	用途
/clock	GZ→ROS	仿真时钟（所有节点的时间源，最重要的一条）
/cmd_vel	ROS→GZ	速度指令下行（唯一一条下行）
/odom	GZ→ROS	轮式里程计
/scan	GZ→ROS	激光
/imu/data	GZ→ROS	IMU（顺手改名：gz 侧叫 /imu）
/camera/image_raw、/camera/camera_info	GZ→ROS	相机（后续课程用）
/joint_states	GZ→ROS	轮子转角（RViz 显示轮子转动）
/tf（gz 侧 /gz/tf）	GZ→ROS	驱动插件发的 odom->base_footprint

4.3 操作：写 Gazebo 后端 launch

src/mecamind_gazebo/launch/gazebo_sim.launch.py 启动三样东西：

```
# ① Gazebo 本体：-r 立即运行，-s 无头模式（use_gui:=false 时）
gz_with_gui = IncludeLaunchDescription(
    PythonLaunchDescriptionSource(f"{ros_gz_share}/launch/gz_sim.launch.py"),
    launch_arguments={"gz_args": ["-r -v 2 ", world]}.items(),
    condition=IfCondition(use_gui),
)

# ② robot_state_publisher：读 URDF 发布机器人骨架 TF
# Command 替换 = launch 启动时执行 `xacro <文件>` 展开成 URDF 字符串
robot_description = Command(
    ["xacro ", f"{description_share}/urdf/mecamind_mecanum.urdf.xacro"]
)

# ③ 桥接器：按 bridge.yaml 翻译话题
Node(
    package="ros_gz_bridge", executable="parameter_bridge",
    parameters=[{"config_file": f"{gazebo_share}/config/bridge.yaml",
                  "use_sim_time": True}],
)
```

再在 mecamind_bringup 里放一个转发入口 gazebo.launch.py，日常使用只需要记 `ros2 launch mecamind_bringup gazebo.launch.py`。

4.4 关于 use_sim_time：全项目最重要的一个参数

仿真世界里的时间由 Gazebo 推进（/clock 话题），可能比真实时间快或慢。**所有节点都必须设 use_sim_time: true**，让它们的时钟跟随 /clock，否则：激光消息带着仿真时间戳、TF 查询用墙钟时间，两边对不上，SLAM/Nav2 会疯狂报 TF 外推错误并丢弃所有数据。

规则很简单：**这个系统里只要有仿真器在跑，每个节点、每个工具（包括 RViz、map_saver）都要带 use_sim_time:=true。**

4.5 验证

```
cd ~/mecamind_ros2 && colcon build && source install/setup.bash
ros2 launch mecamind_bringup gazebo.launch.py

# 另开终端逐项检查：
ros2 topic hz /scan          # ≈10Hz
ros2 topic hz /odom          # ≈30Hz
ros2 topic echo /clock --once # 有仿真时间在走
ros2 run tf2_tools view_frames # 生成 frames.pdf: odom->base_footprint->...->lidar_link

# 用 ROS 话题开车 (teleop 键盘遥控)
ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

4.6 常见错误

现象	原因与解法
/scan 没数据但 Gazebo 里能看到激光	bridge.yaml 类型写错或桥没起来， <code>ros2 node list</code> 查 <code>mecamind_gz_bridge</code>
TF 树断在 base_footprint	驱动插件的 <code><tf_topic>/gz/tf</tf_topic></code> 与 <code>bridge.yaml</code> 不匹配
机器人不动	<code>/cmd_vel</code> 方向写成了 <code>GZ_T0_ROS</code> ；或模型没加载（查 <code>GZ_SIM_RESOURCE_PATH</code> ）

第 5 步：自研轻量 2D 仿真器

军队既要实兵演习，也要沙盘推演。Gazebo 是实兵演习：贴近真实、连轮子打滑都模拟，但开销大、启动慢；本步要造的轻量仿真器就是 沙盘：抓住主要矛盾（位置、激光、时钟），舍弃次要细节（三维 动力学、渲染），换来秒级启动和完全确定的行为——自动化测试和 低配电脑的救星。更重要的是，亲手推一遍沙盘，你才真正明白 "里程计"和"激光"这两路数据是怎么产生的。

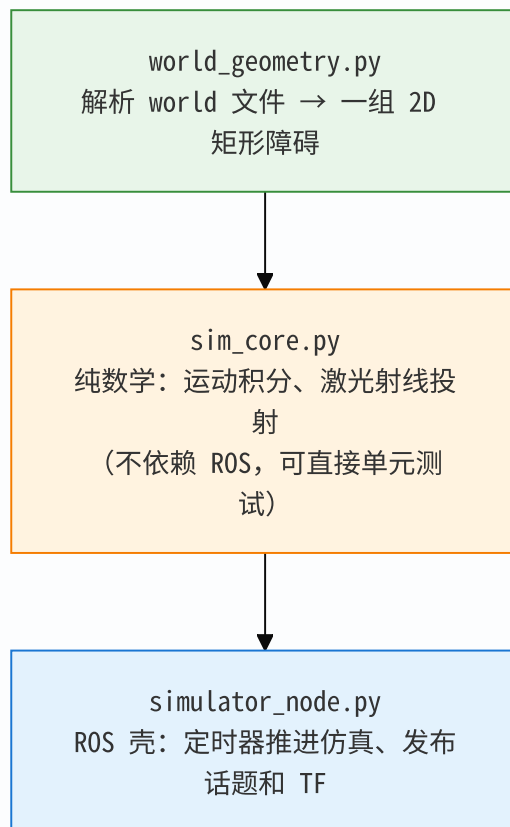
5.1 目标

写一个纯 Python 的 2D 仿真器节点，一个节点顶替 Gazebo 的全部职责：发布 `/clock`、`/scan`、`/odom`、`/imu/data`、TF `odom->base_footprint`，订阅 `/cmd_vel` 驱动机器人。

5.2 为什么要自己写一个仿真器

- Gazebo 吃 CPU/GPU，配置较低的电脑、CI 服务器经常跑不动；
- 无头自动测试（CP2/CP3）需要一个秒级启动、确定性强的后端；
- 教学价值巨大：亲手实现里程计积分和激光射线求交，比调用黑盒更能理解 SLAM 的输入是怎么来的。

代码分三层（都在 `src/mecamind_tools/mecamind_tools/`）：



分层的意义：数学核心 `sim_core.py` 不 import 任何 ROS 模块，`pytest` 可以毫秒级反复验证它；ROS 相关的胶水都挤到最外层。

5.3 原理 ①：世界几何复用

轻量仿真直接解析 `worlds/three_room_house.world` (SDF 文本)，读取每个 link 的 `pose` + `box` 尺寸，投影成 2D 矩形列表。墙体坐标与 Gazebo 世界完全一致，两个后端"看到"同一个环境。

5.4 原理 ②：里程计积分（麦轮全向运动学）

机器人收到的 `/cmd_vel` 是本体坐标系速度 (`vx` 前、`vy` 左、`wz` 逆时针)。把它积分成世界坐标位姿，需要旋转矩阵变换：

$$\begin{aligned}x(t+dt) &= x + (vx \cdot \cos(yaw) - vy \cdot \sin(yaw)) \cdot dt \\y(t+dt) &= y + (vx \cdot \sin(yaw) + vy \cdot \cos(yaw)) \cdot dt \\yaw(t+dt) &= yaw + wz \cdot dt\end{aligned}$$

麦轮车 $vy \neq 0$ (能横移)，所以两个公式都有两项——这是全向底盘 与 差速底盘 (vy 恒 0) 在数学上的唯一区别。

碰撞处理用"候选位姿降级"策略：先试全速位移，撞墙则尝试只保留 x 或 y 分量 (滑墙效果)，都不行就原地不动。

5.5 原理 ③：激光射线投射 (slab 法求交)

对 360 个方向逐束计算：从机器人位置出发，沿 `角度 = 雷达坐标系角度 + 机器人 yaw` 的方向发射射线，与每个矩形用 **slab 法** 求交（把矩形拆成 x/y 两组平行边界，求射线进入/离开每组的参数区间，区间有交集则命中），取所有矩形中最近的交点距离作为该束的量程。

再叠加两样"真实感"：

- **高斯噪声** (`lidar_noise_std`，默认 0.01m)：不只是拟真——完美对齐栅格边界的激光会让 SLAM 永远无法把某些墙格标记为占据（第 8 步的坑），一点噪声反而让建图更稳定；
- **随机丢包** (`lidar_dropout_prob`)：模拟真实雷达的无效点。

5.6 原理 ④：仿真时钟

节点内部维护仿真时间，每个 tick 递增 `dt` 并发布到 `/clock`。这样轻量后端与 Gazebo 后端对下游完全一致——下游节点都设 `use_sim_time: true`，根本不知道时钟是谁发的。

5.7 代码精讲：运动积分与"滑墙" (`sim_core.py` 摘录)

`_integrate_candidate` 是整个仿真器最核心的十几行，逐行理解：

```
# 本体速度旋转到世界坐标系 (2D 旋转矩阵 R(yaw) 乘速度向量的展开式)
dx = (self.velocity.vx * math.cos(base_yaw) - self.velocity.vy * math.sin(base_yaw)) * dt
dy = (self.velocity.vx * math.sin(base_yaw) + self.velocity.vy * math.cos(base_yaw)) * dt
# 朝向积分后立刻归一化，防止长时间旋转导致角度无界增长
yaw = _normalize_angle(base_yaw + self.velocity.wz * dt)

# 候选按"完整移动 > 只沿 x 滑动 > 只沿 y 滑动 > 原地转向"降级尝试
candidates = [
    (base_x + dx, base_y + dy, yaw),
    (base_x + dx, base_y, yaw),
    (base_x, base_y + dy, yaw),
    (base_x, base_y, yaw),
]
for new_x, new_y, new_yaw in candidates:
    if not self._collision(new_x, new_y):
        self.pose = SimPose(new_x, new_y, new_yaw)
        return
```

三个值得学习的细节：

- **角度归一化**用 `atan2(sin, cos)` 而不是 `while` 循环减 2π ——`sin/cos` 天然消除整圈数，数值上最稳；
- **候选降级**实现了"贴墙滑动"的手感：斜着撞墙时，撞的方向被取消，另一个方向照常走，而不是整个卡死；
- **碰撞检测**把机器人简化成半径 0.22m 的圆，等价于"把障碍向外膨胀一个机器人半径、把机器人当质点"——这是运动规划里经典的配置空间 (C-space) 技巧，第 12 步代价地图的 `inflation` 是同一思想。

激光那边，`scan_ranges` 的关键一行是每束光的世界朝向角：

```
heading = self.pose.yaw + self.lidar_angle_min + index * increment
```

LaserScan 的角度定义在**雷达自身坐标系**里 (angle_min 到 angle_max)， 所以必须加上机器人当前 yaw 才是世界方向——很多人自己写仿真器 时忘了这一步，得到的点云会随机器人旋转而"甩尾"。

另外注意仿真器还做了**速度斜坡** (_ramp)：速度不是瞬间跳到 指令值，而是以 $\text{max_accel} \times dt$ 为单步上限逐渐逼近，模拟电机 "不能瞬间变速"的物理事实（梯形速度曲线）。

5.8 操作与验证

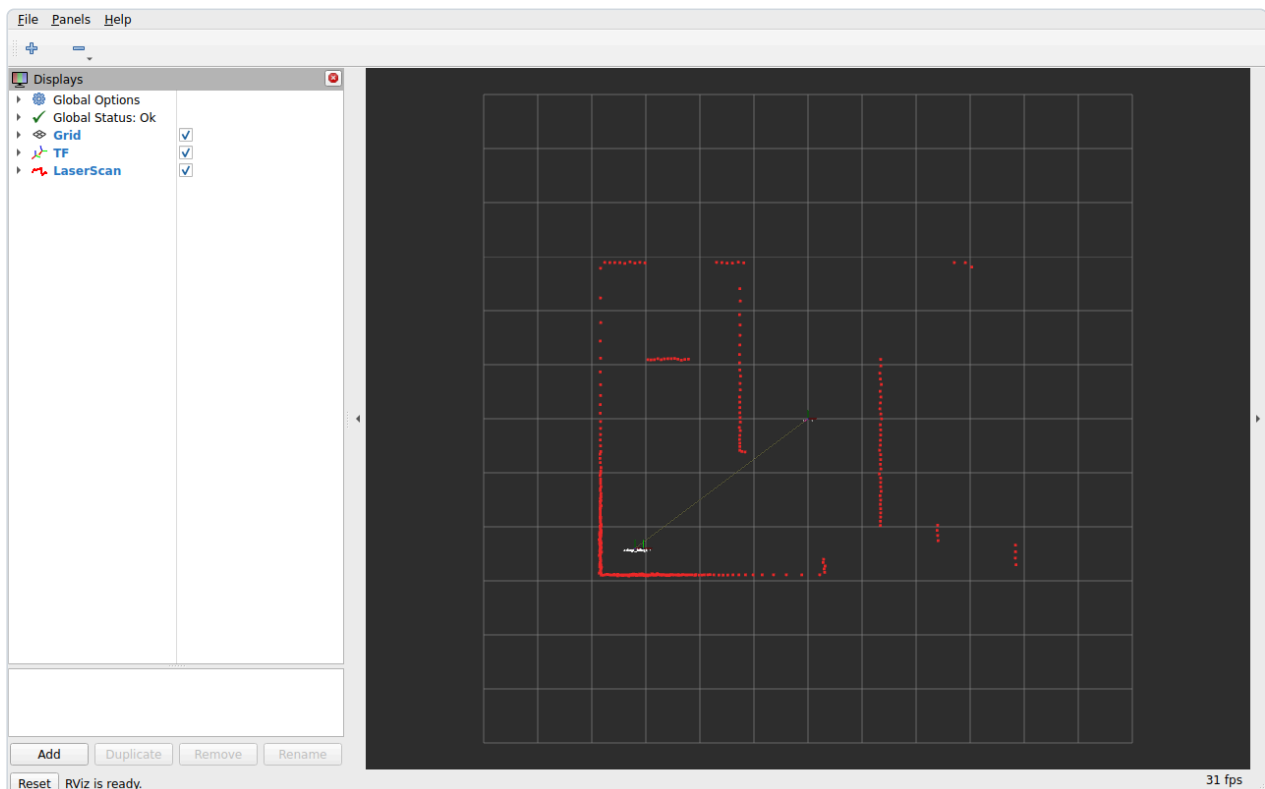
launch 文件 `src/mecamind_bringup/launch/lite_sim.launch.py` 只起 这一个节点，注意**话题重映射**——节点内部用 `/scan_raw` 等原始名， launch 里统一映射成全项目约定名：

```
remappings=[
    ("/scan_raw", "/scan"),
    ("/imu/data_raw", "/imu/data"),
    ("/controller/cmd_vel", "/cmd_vel"),
],
```

验证方法与 Gazebo 后端一模一样（这正是统一接口的意义）：

```
ros2 launch mecamind_bringup lite_sim.launch.py
ros2 topic hz /scan && ros2 topic hz /odom
ros2 run teleop_twist_keyboard teleop_twist_keyboard # 能开动
```

在 RViz 里订阅 `/scan`（Fixed Frame 选 `odom`）可以直观检验射线 投射的正确性：红色激光点整齐地勾勒出三房间的墙壁和家具轮廓， 和 Gazebo 后端扫出来的形状一致——说明 `world_geometry` 与 Gazebo 世界确实对齐了：



轻量仿真器发布的激光扫描：红点勾勒出与 Gazebo 世界一致的房间轮廓

5.9 动手练习

- 1. 给仿真器节点传 `-p odom_xy_noise_std:=0.01`，再用 `ros2 run tf2_ros tf2_echo odom base_footprint` 观察里程计 开始"抖"——这是第 14 步漂移实验在 lite 后端的做法；
- 2. 把 `lidar_beams` 改成 90（低线数雷达），观察 RViz 里激光 点变稀疏——想想这对第 8 步的建图会有什么影响（提示：8.5 的坑 会更早出现）；
- 3. 读 `sim_core.py` 的 `_ramp` 函数，回答：如果去掉速度斜坡（直接 `velocity = command`），遥控手感会有什么变化？

第 6 步：统一接口——双后端可互换的关键

任何一台电器插进任何一个插座都能通电，靠的不是电器认识插座，而是双方共同遵守国标：孔距、电压、频率都白纸黑字定死。本步做的就是给两个仿真后端定"国标"：话题名、消息类型、TF 树、出生点 全部统一。之后 SLAM、Nav2 这些"电器"只对国标编程，根本不需要 知道墙后面接的是 Gazebo 还是轻量仿真器。

6.1 目标

把"两个后端对外长得一模一样"这件事固化成明确的契约，后面所有模块（SLAM、Nav2、工具）都只对契约编程。

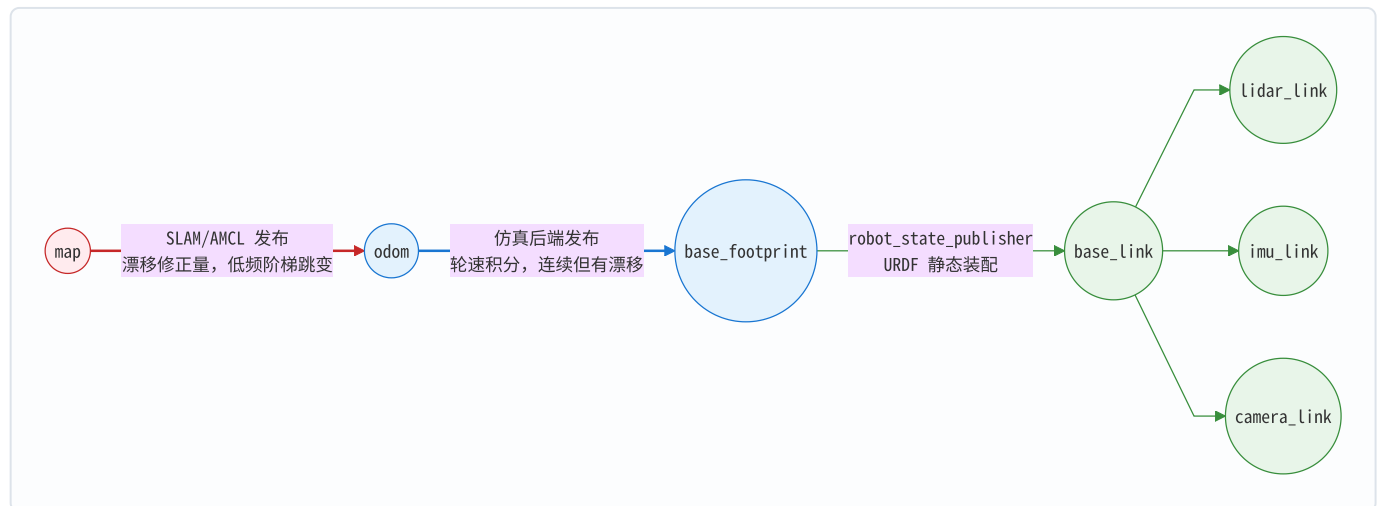
6.2 接口契约

接口	类型	方向	说明
<code>/clock</code>	<code>rosgraph_msgs/Clock</code>	后端 → 全体	仿真时间源
<code>/scan</code>	<code>sensor_msgs/LaserScan</code>	后端 → 感知	360 束， <code>frame_id= lidar_link</code>
<code>/odom</code>	<code>nav_msgs/Odometry</code>	后端 → 需要者	轮式里程计（Gazebo 的带真实漂移）
<code>/cmd_vel</code>	<code>geometry_msgs/Twist</code>	控制 → 后端	最终执行的速度
TF <code>odom→base_footprint</code>	<code>tf2</code>	后端 → 全体	里程计位姿
TF <code>base_footprint→...→lidar_link</code>	<code>tf2</code>	<code>robot_state_publisher</code>	机器人骨架（静态）

配套的**统一约定**还有两条：

- 1. **出生点统一在 origin (0,0,0)**。Gazebo 模型的 `<pose>` 和 lite 的 `initial_x/y/yaw` 都设为原点。这样第 8 步的建图路线文件、第 12 步的初始位姿配置，两个后端可以共用同一份。
- 2. `use_sim_time: true` 全员生效（见 4.4）。

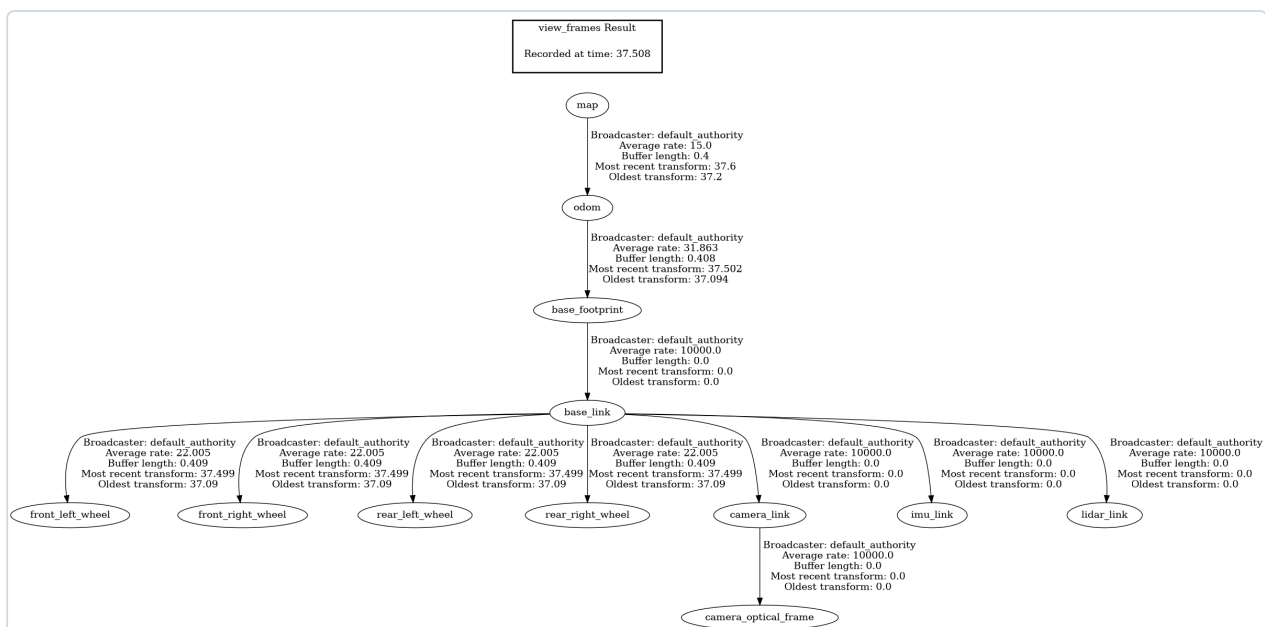
6.3 TF 树的完整形态（提前理解，后面反复用）



红色一段由定位模块负责，蓝色一段由后端里程计负责，绿色部分是 URDF 定死的机器人骨架——三种颜色对应三个发布者，谁出问题 查谁。

为什么拆成 map→odom 和 odom→base 两段？因为两类使用者需求矛盾：控制回路需要**连续不跳变**的位姿（用 odom 系），规划器需要**绝对准确**的位姿（用 map 系）。把"修正量"放在 map→odom 这一段，两边都满意。这是 ROS 导航体系最精妙的设计，详见 docs/10.3.3_专题_里程计漂移与矫正.md。

下面是建图运行时用 `ros2 run tf2_tools view_frames` 导出的真实 TF 树。注意每段变换的发布频率差异：
map→odom 只有 15Hz（SLAM 低频修正），odom→base_footprint 约 32Hz（里程计连续发布）；base_link 往下，四个轮子约 22Hz（随 /joint_states 转动），雷达/IMU/相机则是静态变换（URDF 固定装配，发一次锁存）：



实际运行时的 TF 树：map→odom→base_footprint→base_link→各传感器

6.4 QoS——话题的"服务质量"合同

统一接口还隐含着 QoS（Quality of Service）的匹配问题，这里一次讲清，后面遇到就不慌了。ROS 2 的每个发布者/订阅者都带一份 QoS 配置，**双方不兼容就收不到消息且往往不报错**——这是 ROS 2 新手 排查话题"明明有发布者却收不到"的第一嫌疑人。本项目用到两种组合：

组合	配置	用在哪
传感器流	BEST_EFFORT + 小队列	/scan、/odom：丢一两帧无所谓，实时性优先
地图/静态数据	RELIABLE + TRANSIENT_LOCAL + depth 1	/map、/robot_description：晚加入的订阅者也要能拿到最后一帧

TRANSIENT_LOCAL（锁存）值得展开：`/map` 更新很慢（几秒一帧），RViz 若在两帧之间启动，普通 QoS 下要干等下一帧才有图；锁存让发布者把最后一帧留着，新订阅者一连上立刻补发。第 9 步 `map_saver_cli` 的兜底参数 `map_subscribe_transient_local:=true` 就是在声明"我要接收锁存消息"。

命令行诊断 QoS：

```
ros2 topic info -v /map      # 能看到双方的 Reliability / Durability
```

6.5 验证：接口一致性检查

两个后端分别启动，跑同一组命令，输出结构应当一致：

```
ros2 topic list | sort      # 核心话题名集合相同
ros2 topic info -v /scan   # 类型、发布者数相同
ros2 run tf2_tools view_frames # TF 树结构相同
```

第 7 步：安全门——速度链路上的保险丝

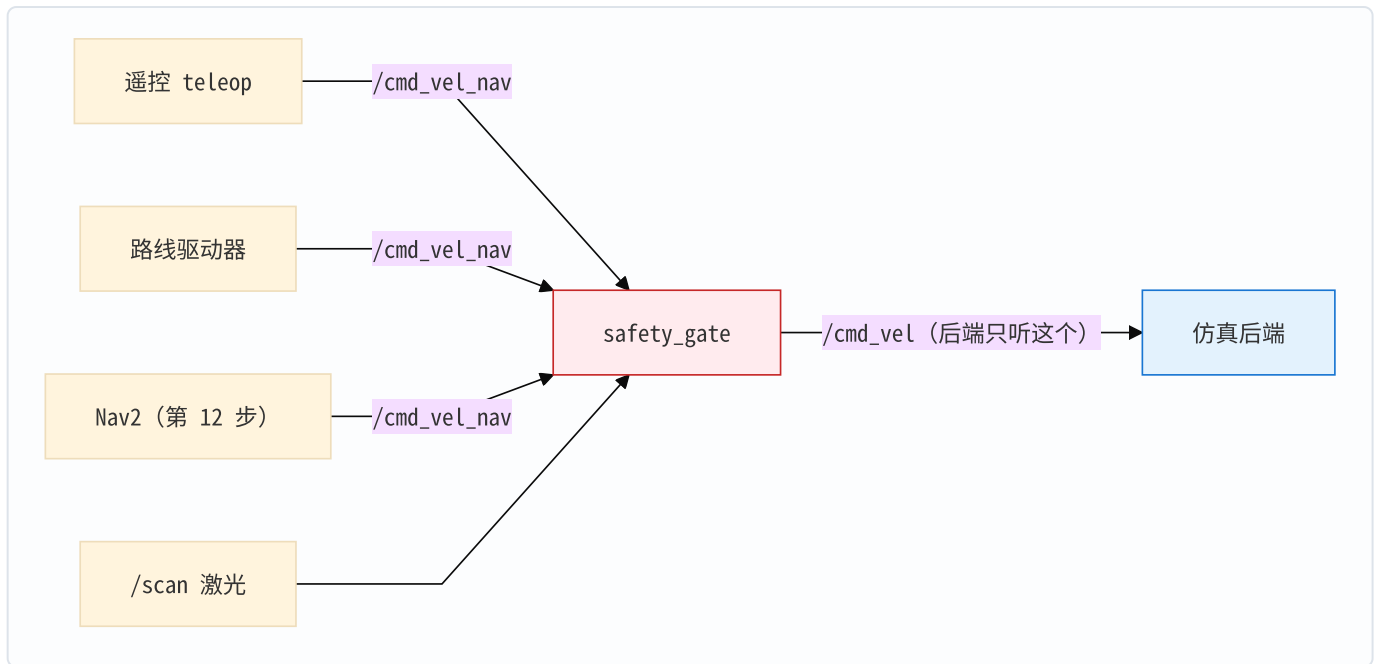
现代汽车的 AEB（自动紧急制动）是个很贴切的参照：不管方向盘后面坐的是新手还是老司机，也不管导航软件规划得对不对，雷达探到前方障碍过近，AEB 都会压低车速直至刹停——它不信任任何上游，只看传感器说话。本步的安全门就是给机器人装 AEB：所有速度指令（遥控、路线驱动器、以后的 Nav2）都要过它这一关。

7.1 目标

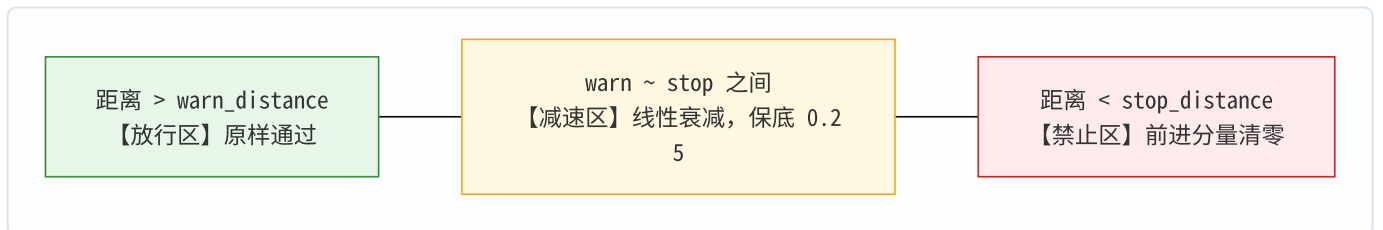
写 `safety_layer.py`（注册为 `mecamind_safety_gate`）：任何上游想开车，都必须经过它的激光避障检查。

7.2 原理：为什么要拦一道

上游（遥控、路线驱动器、Nav2）都可能发出危险速度。与其相信每个上游都写对，不如在链路末端统一设一道关卡——这就是"纵深防御"。约定：



安全门内部按距离分三个区间处理前进速度：



7.3 核心逻辑 (src/mecamind_tools/mecamind_tools/safety_layer.py)

1. **扇区最近距离**：取激光正前方一个扇区的最近值（LaserScan 的下标 i 对应角度 $\text{angle_min} + i \cdot \text{increment}$ ）；
2. **两级阈值**：
 - 距离 $< \text{stop_distance}$ → 前进分量清零（只拦前进，倒车和旋转放行，否则车会卡死在墙边无法脱困）；
 - $\text{stop} \sim \text{warn}$ 之间 → 前进速度按比例线性衰减；
 - 距离 $> \text{warn_distance}$ → 原样放行；
3. **两个看门狗**：激光超时（`scan_timeout_sec`）或上游指令超时（`cmd_timeout_sec`）都立即发零速。传感器瞎了或大脑失联时，正确行为是停车而不是维持旧速度。

实战调参教训：Gazebo 在 WSL2 上 RTF（实时因子）常低于 1，激光按墙钟看间隔会变长，超时阈值设太紧会被误判 `scan_stale` 而频繁刹车。本项目把 `scan_timeout_sec` 放宽到 2.0s、`cmd_timeout_sec` 1.5s（见 `mapping.launch.py` 里的注释）。

7.4 代码精讲：gate_twist——"修剪"而不是"否决"

安全门对速度的处理体现了很细腻的取舍（`safety_layer.py` 摘录）：


```
def gate_twist(command: Twist, decision: SafetyDecision) -> Twist:
    gated = Twist()
    gated.linear.x = command.linear.x
    gated.linear.y = command.linear.y
    gated.angular.z = command.angular.z          # 旋转从不限制

    if command.linear.x > 0.0:
        # 只有"前进"分量按 scale_x 缩放 (stop 区时 scale_x=0 即截断)
        gated.linear.x = command.linear.x * decision.scale_x
    # 横移方向: 先看硬禁止 (对应侧已到 stop 线), 再看软衰减
    if command.linear.y > 0.0 and not decision.allow_left:
        gated.linear.y = 0.0
    elif command.linear.y < 0.0 and not decision.allow_right:
        gated.linear.y = 0.0
    else:
        gated.linear.y = command.linear.y * decision.scale_y
    return gated
```

设计要点逐条拆解:

- **倒车不限** (只处理 `linear.x > 0`) : 车被堵在墙边时必须还能退出来, 全部否决会造成永久卡死;
- **旋转不限**: 原地旋转不产生平移位移, 且往往是脱困所必需的动作;
- **横移分侧管** (麦轮特有): 左移看左侧障碍, 右移看右侧, 互不干涉;
- **warn 区线性衰减但保底 0.25**: 临界处若无保底, 车会慢到几乎不动, 看起来像卡死, 反而更难排查;
- **函数无副作用** (不修改传入的 `command`), 返回新对象——写数据处理函数的好习惯, 方便单元测试。

`analyze_scan` 则负责把一帧激光变成决策: 取前/左/右/后四个扇区 (`LaserScan` 下标 `i` 对应角度 `angle_min + i * increment`) 的最近距离, 与 `stop/warn` 两级阈值比较。这两个函数都是纯函数, `test_mecamind.py` 里对它们有完整的单元测试。

7.5 验证

```
ros2 launch mecamind_bringup lite_sim.launch.py &
ros2 run mecamind_tools mecamind_safety_gate --ros-args \
  -p use_sim_time:=true -p scan_topic:=scan \
  -p input_cmd_topic:=/cmd_vel_nav -p output_cmd_topic:=/cmd_vel &

# 朝墙发全速前进指令, 观察 /cmd_vel 被衰减/清零
ros2 topic pub -r 10 /cmd_vel_nav geometry_msgs/msg/Twist '{linear: {x: 0.5}}' &
ros2 topic echo /cmd_vel          # 靠近墙时 linear.x 应逐渐变小直至 0
```

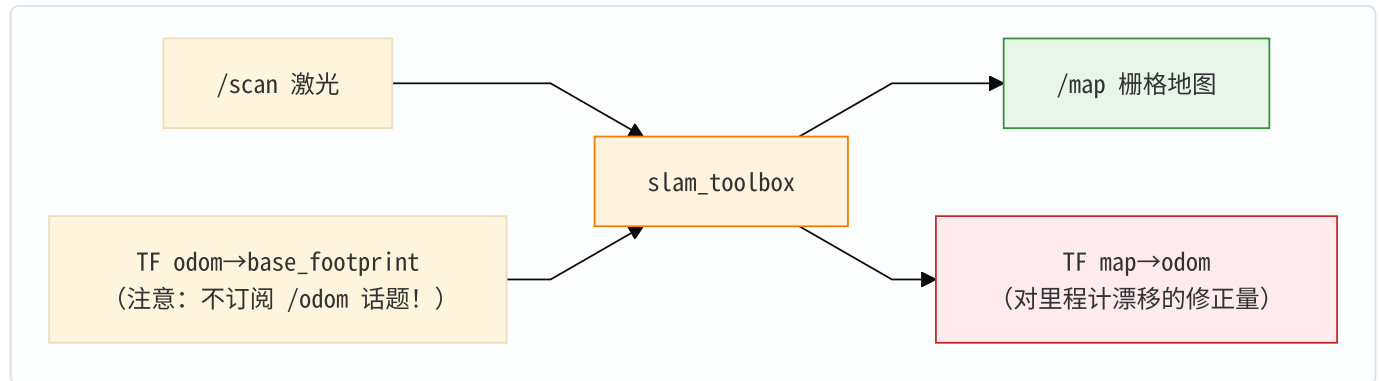
第 8 步: 接入 SLAM Toolbox 建图

想象你被蒙住眼睛带进一栋陌生房子, 手里只有一根盲杖: 敲一下, 探到一堵墙; 走两步, 再敲——你在脑子里慢慢拼出户型图。但有个麻烦: 画图的前提是知道自己走到了哪, 而判断自己走到了哪又得靠已经画出的图。"定位"和"建图"互为因果、必须同时进行, 这就是 SLAM (Simultaneous Localization And Mapping) 里 "Simultaneous" 的含义。机器人的盲杖豪华一些: 一圈 360 束的激光。

8.1 目标

接入 `slam_toolbox`，遥控开车就能在 RViz 里看到地图逐渐长出来，一条命令 `ros2 launch mecamind_bringup mapping.launch.py backend:=gazebo|lite` 拉起完整建图系统。

8.2 原理：SLAM Toolbox 的输入输出



核心算法是**扫描匹配** (correlative scan matching)：拿新一帧激光 与已建地图对比，以里程计预测位姿为中心，在一个小窗口内穷举搜索 "激光与地图重合度最高"的位姿。里程计负责短期预测（缩小搜索范围），激光负责长期纠偏（消除漂移）。

常见误区：很多人以为 SLAM 订阅 `/odom` 话题。用 `ros2 topic info -v /odom` 可以证明 `slam_toolbox` 不在订阅者列表里——它是从 TF 树上查 `odom->base_footprint` 变换的。

8.3 操作 ①：SLAM 参数文件

`src/mecamind_tools/config/mecamind_slam_toolbox_params.yaml` 的关键项（文件里每一项都有注释，这里只讲三组必须理解的）：

```
slam_toolbox:
  ros_parameters:
    mode: mapping          # 建新图（另有 localization 模式）
    odom_frame: odom       # 三个坐标系名与全项目约定一致
    map_frame: map
    base_frame: base_footprint
    scan_topic: /scan
    resolution: 0.05       # 5cm/格，室内常用值
    max_laser_range: 8.0   # 与雷达实际量程一致
    min_pass_through: 1    # ★ 本项目最重要的一个调参，见 8.5
    minimum_travel_distance: 0.10 # 挪 10cm 才处理新帧：静止时不重复累积
    do_loop_closing: true  # 回环检测：回到走过的地方时消除累积误差
```

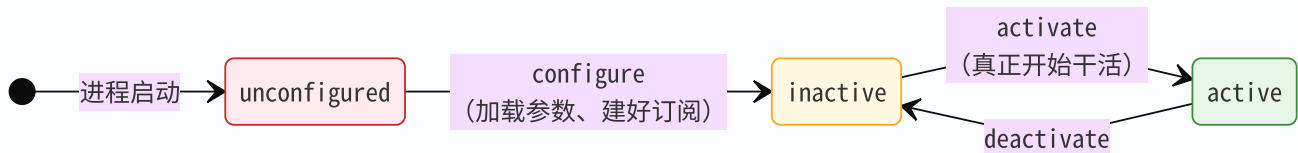
8.4 操作 ②：统一建图 launch

`src/mecamind_bringup/launch/mapping.launch.py` 的组装逻辑：

1. 按 `backend` 参数二选一地 include 后端 launch（`IfCondition` + `PythonExpression` 实现 launch 里的 `if/else`）；

2. 起 `slam_toolbox` (异步版 `async_slam_toolbox_node`);
3. **`slam_toolbox` 是 lifecycle 节点**——启动后停在 `unconfigured` 状态 不干活, 须有人调 `configure`→`activate`。我们写了个通用的 `mecamind_lifecycle_activator` 节点, 延迟 4 秒 (等仿真时钟先跑起来) 把它激活;
4. 起安全门 (第 7 步);
5. 可选起 RViz (`with_rviz:=false` 用于无头测试)。

关于 lifecycle 节点 (第 12 步还会大量遇到, 这里详细说明): 普通节点一启动就工作, lifecycle 节点有显式状态机:

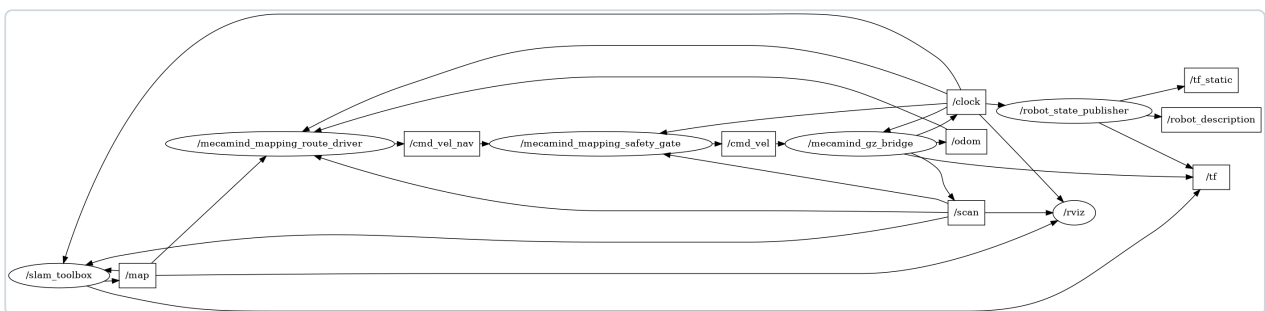


好处是启动顺序可控——比如"先让 `map_server` 加载好地图, 再让 `AMCL` 激活"。我们的激活器 `lifecycle_activator.py` 就是按列表 依次调每个节点的 `change_state` 服务; 它先查当前状态再补缺的 迁移 (幂等, 重复运行无害)。手动操作等价于:

```

ros2 lifecycle set /slam_toolbox configure
ros2 lifecycle set /slam_toolbox activate
ros2 lifecycle get /slam_toolbox    # 应显示 active
  
```

建图系统全部起来后的节点图如下 (`rqt_graph` 导出)。对照第 6 步 的接口契约看: 仿真后端 (图中是 `mecamind_gz_bridge`) 发出 `/scan`、`/odom`、`/clock`, `slam_toolbox` 消费 `/scan` 和 TF 产出 `/map`; 速度指令则沿 `/cmd_vel_nav` → 安全门 → `/cmd_vel` 流回后端:



建图系统运行时的节点与话题连接图

8.5 踩坑实录 ①: 地图上的墙缺了两面

这是本项目印象最深的一个坑, 完整排查过程值得复盘:

- **现象:** lite 后端建出的图 north/east 两面墙时有时无, 质量分析报 `missing_walls`。
- **排查:** 机器人明明扫到了那两面墙 (`/scan` 数据正常), 但 `/map` 上对应格子一直是未知。
- **根因:** 360 束激光在距离 >1.4m 处相邻光束的横向间距超过一个 5cm 栅格。slam_toolbox 默认 `min_pass_through: 2` 要求一个格子 被命中 2 次才标记为占据——远处的墙格常常只被 1 束扫到, 永远攒不够 2 次。

- **修复**: `min_pass_through: 1` (单次命中即计入), 再给 `lite` 后端 加 `0.01m` 激光噪声, 避免激光反复打在完全相同的位置。

方法论: 建图缺墙时, 先算一下 `距离 × 角分辨率` 与栅格尺寸的关系, 判断是不是命中密度不足。

8.6 踩坑实录 ②: TF_OLD_DATA 刷屏

- **现象**: `Ctrl+C` 重启 `launch` 后, `slam_toolbox` 和 `RViz` 疯狂报 `TF_OLD_DATA ignoring data from the past`。
- **根因**: 上一次的 `gz sim` 进程没死透, 还在发旧仿真时间的 `/clock` 和 `TF`; 新 `launch` 从仿真时间 0 重新开始, 时间线冲突。
- **修复**: 写了清理脚本, 重启前一律执行:

```
bash scripts/cleanup_ros2.sh --all # 杀干净 gz sim / rviz / slam 残留进程
```

8.7 RViz 使用入门

`RViz` 是 `ROS` 的通用可视化工具, 本项目预置了布局文件 (`src/mecamind_tools/rviz/mecamind_mapping.rviz`), `launch` 会用 `-d` 自动加载。但你应该会自己从零配一遍:

1. **Fixed Frame** (左侧 Global Options): 一切显示的参考系。建图时选 `map`; 如果选了不存在的 `frame`, 所有显示都会报错——这是 `RViz` 最高频的新手问题;
2. **Add 按钮** 添加显示项, 本步用到:
 - `Map` (topic 选 `/map`): `SLAM` 的产出;
 - `LaserScan` (topic 选 `/scan`): 激光点, 建议 `Size` 调大到 `0.05` 方便看清;
 - `TF`: 坐标轴可视化, 调 `Marker Scale` 避免太乱;
 - `RobotModel` (Description Topic 选 `/robot_description`): 车模型;
3. **视角**: 鼠标左键旋转、中键平移、滚轮缩放。俯视图最适合建图 (Views 面板切 `TopDownOrtho`);
4. 配好的布局用 `File` → `Save Config As` 存成 `.rviz` 文件, 下次 `-d` 加载——我们仓库里的预置布局就是这么来的。

`WSLg` 下 `RViz` 窗口偶尔会撑满屏幕拖不动: 按 `Win+↓` 还原, 或用 `python3 scripts/resize_window.py` 强制改尺寸。

8.8 验证：手动建图全流程

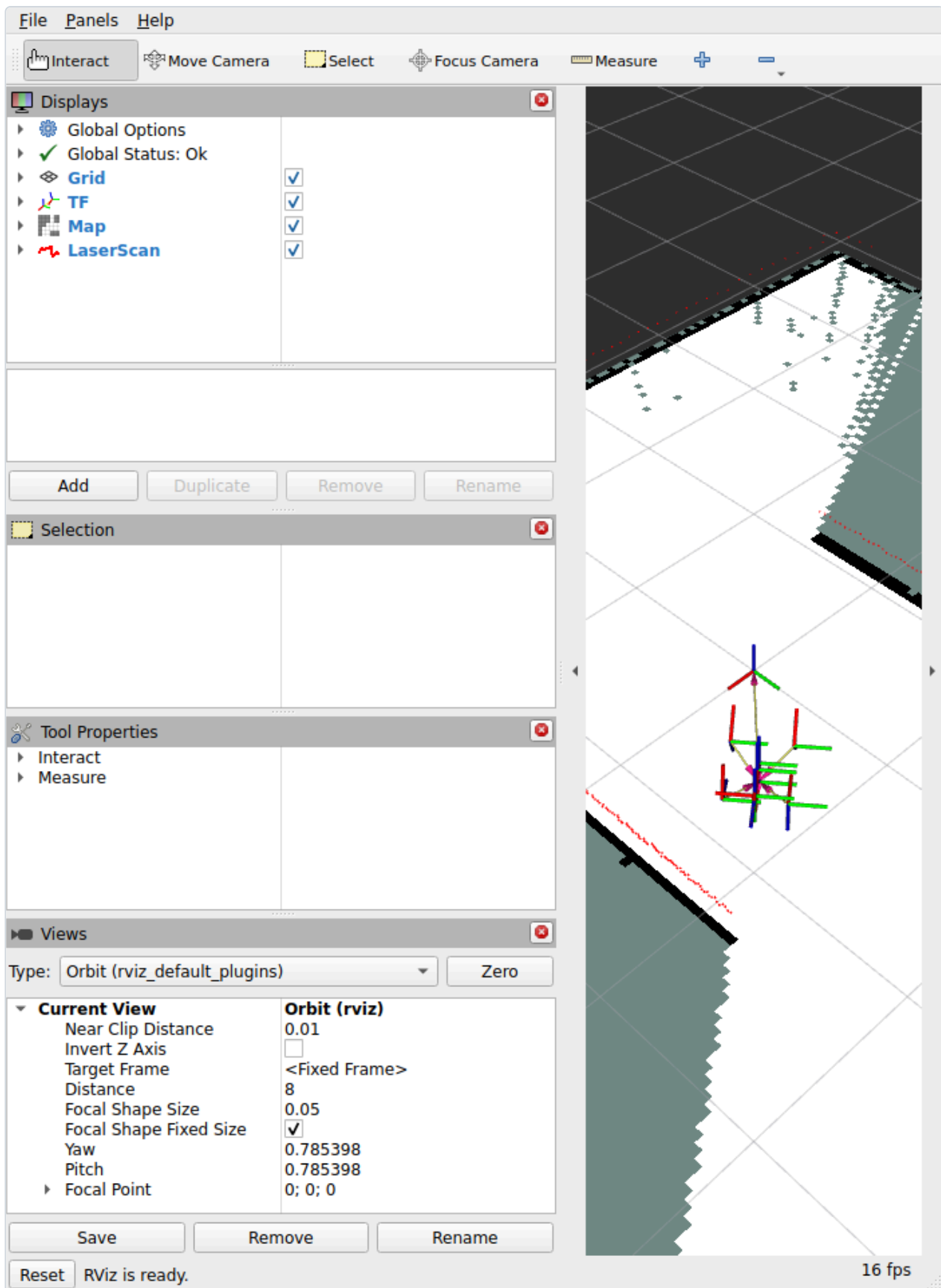
```
bash scripts/cleanup_ros2.sh --all
ros2 launch mecanind_bringup mapping.launch.py backend:=gazebo # 或 lite

# 另开终端遥控开车，绕房间一圈
ros2 run teleop_twist_keyboard teleop_twist_keyboard \
  --ros-args -r /cmd_vel:=/cmd_vel_nav # 注意发给安全门的入口话题

# RViz 自检三项：白色自由区随车扩展；激光红点贴在黑色墙上；
# 画面全空白就检查 Fixed Frame 是否为 map

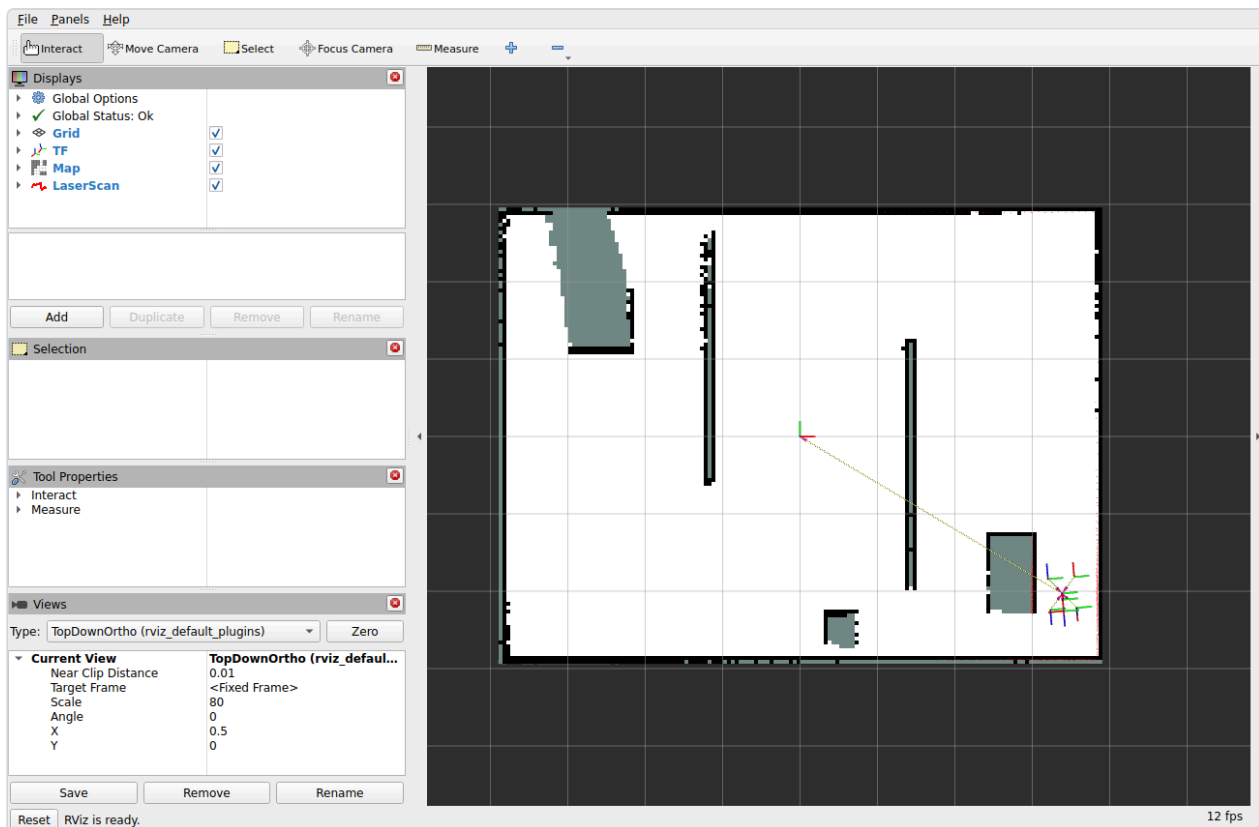
# 建完存图（use_sim_time 必须带！lite 后端不带会订阅失败）
ros2 run nav2_map_server map_saver_cli -f maps/my_first_map \
  --ros-args -p use_sim_time:=true
```

建图过程的两个阶段对比。刚起步时只有机器人周围一小片被探明（白=自由，黑=占据，灰/深色=未知），激光红点打在墙面上：



建图初期：只有机器人附近区域被探明，大片区域仍是未知

绕完一圈后，整个三房间轮廓完整浮现，四面外墙闭合、内部隔断 和家具都已画实——这样的图才能进入下一步的质量分析：



建图完成：三房间轮廓完整、外墙闭合

产物是一对文件：`my_first_map.pgm`（图像）+ `my_first_map.yaml`（元信息：分辨率、原点、阈值）。

8.9 动手练习

1. 用 `ros2 topic info -v /odom` 验证 8.2 的说法：`slam_toolbox` 真的不在 `/odom` 的订阅者列表里；
2. 建图途中执行 `ros2 run tf2_ros tf2_echo map odom`——lite 后端 里程计完美，这个变换应接近零；换 Gazebo 后端再看，它会随 行驶缓慢变化（这就是 SLAM 在修正漂移）；
3. 把 `minimum_travel_distance` 改成 1.0（挪 1 米才处理一帧），重新建图，观察地图更新明显变“卡顿”——理解这个参数是 数据量与实时性的旋钮。

第 9 步：半自动建图——路线驱动器

扫地机器人第一次到你家，会先自己绕着屋子转一圈，把户型摸清 存成地图，之后的清扫全靠这张图。本步就是给我们的机器人装上 这个“第一次巡屋”的能力：预先设计一条覆盖全屋、还刻意折返 制造回环的路线，机器人自动跑完、自动存图、自动关闭全部程序 ——从此建图不再需要人握着键盘遥控。

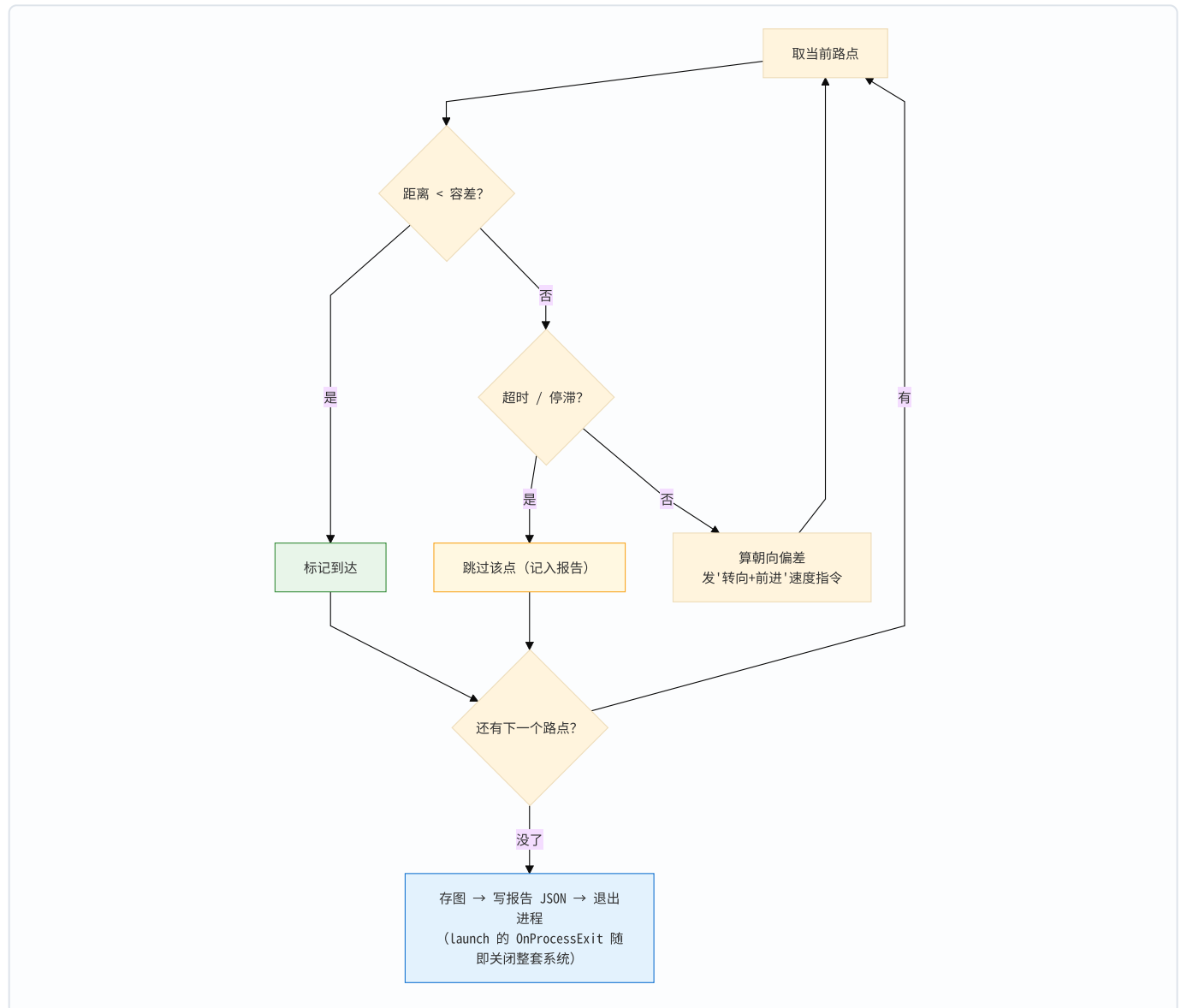
9.1 目标

写 `mapping_route_driver.py`：机器人按预设路点自动绕场建图，跑完自动存图、写路线报告、整套系统自动关闭。全程无人值守。

9.2 原理 ①：路线文件与推进状态机

路线 `src/mecamind_tools/config/mecamind_mapping_route.yaml` 是 9 个世界坐标路点，设计成覆盖三个房间且有意折返制造回环。

驱动器的主循环（`_tick`）是个简单状态机：



launch 里注册了 `OnProcessExit` 事件：驱动器进程一退出就向整个 launch 发 Shutdown，把仿真、SLAM、RViz 全部收掉——这是自动验收 能"跑完自动结束"的机关。

9.3 原理 ②：踩坑实录——必须用仿真时间做超时判断

第一版驱动器用 `time.time()`（墙钟）判断超时，在 lite 后端一切正常，换 Gazebo 后灾难性失败：9 个路点只能到 1~3 个，满屏 `no progress`。

根因：Gazebo 在 WSL2 上 RTF ≈ 0.5 ，即仿真 1 秒要花墙钟 2 秒。机器人按仿真时间正常开着，墙钟却快进两倍——驱动器冤枉它"停滞"。

修复：全部时间判断改用节点时钟（跟随 `/clock` 的仿真时间）：


```
now = self.get_clock().now().nanoseconds * 1e-9 # 而不是 time.time()
```

方法论：写与仿真交互的节点，任何超时/限速逻辑一律用 `node.get_clock()`，它会自动跟随 `use_sim_time`。

9.4 原理 ③：踩坑实录——直接把 OccupancyGrid 写成 PGM

第二个坑：路线跑完后调 `map_saver_cli` 存图，在 `sim time` 下频繁报 `Failed to spin map subscription`（临时节点等不到锁存消息就超时退出）。

修复：驱动器自己订阅 `/map`，缓存最新一帧 `OccupancyGrid`，路线结束时直接写 `PGM/YAML` 文件，`map_saver_cli` 降级为备用方案。写文件要注意两个约定（第 10 步还会遇到它们的镜像问题）：

- 像素值映射：占据(100)→0 黑，空闲(0)→254 白，未知(-1)→205 灰；
- 行序翻转：OccupancyGrid 的 row 0 在 `y_min`（下边），PGM 的 row 0 在图像顶部（`y_max`），写文件时要垂直翻转。

代码精讲（`mapping_route_driver.py` 摘录）——Y-flip 就藏在一行里：

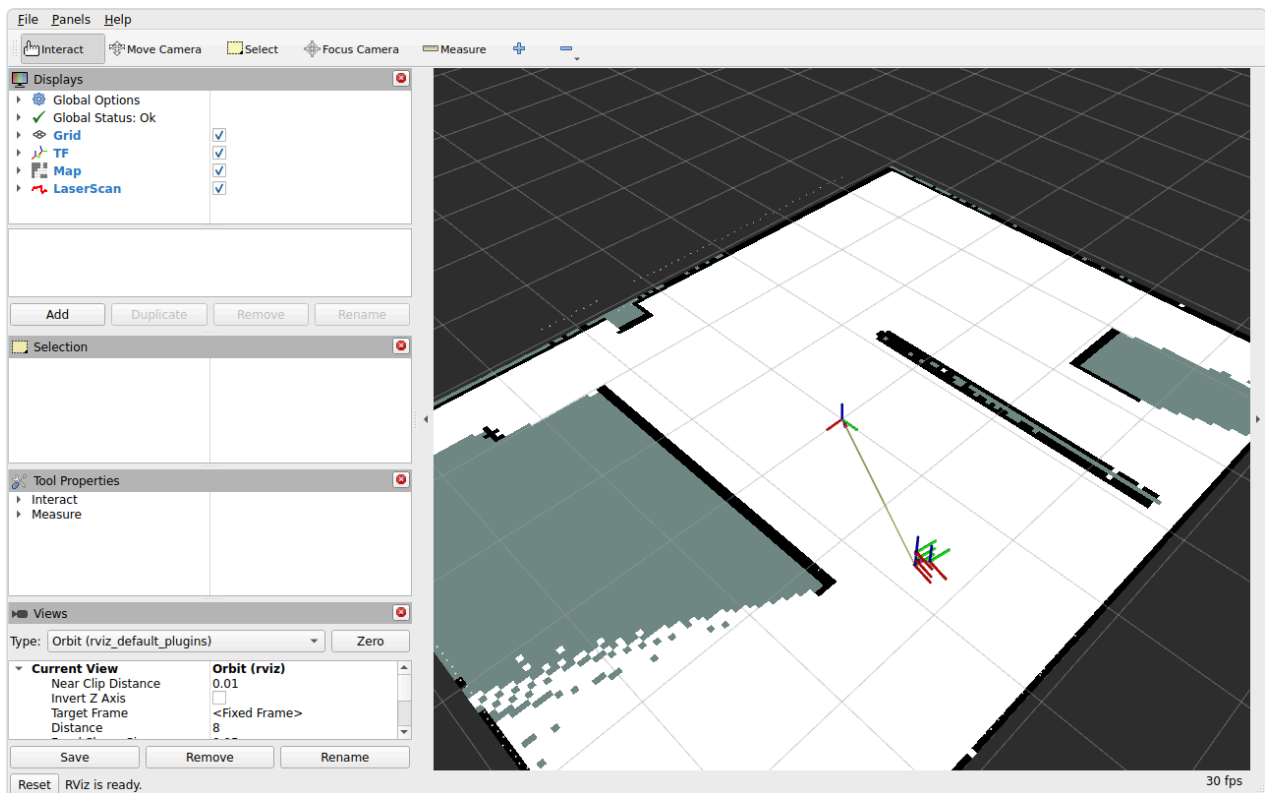
```
pixels = bytearray(width * height)
for row in range(height):
    src_row = height - 1 - row # ★ Y-flip: PGM 第 row 行取自栅格倒数第 row 行
    for col in range(width):
        value = msg.data[src_row * width + col] # row-major 一维数组取值
        if value == -1:
            pixels[row * width + col] = 205 # 未知 → 灰
        elif value >= 65:
            pixels[row * width + col] = 0 # 占据 → 黑
        else:
            pixels[row * width + col] = 254 # 空闲 → 白
```

配套的存图策略是“直接写文件优先、`map_saver_cli` 兜底”（`_try_save_map`）：本节点自己一直订阅着 `/map`，写文件时用的是缓存的最新一帧，**不需要新起任何进程**，“新订阅者收不到锁存消息”的时序问题就从根上不存在了。只有极少数情况下（本节点自己也还没收到过地图）才退回官方工具重试 3 次。

9.5 验证

```
bash scripts/cleanup_ros2.sh --all
ros2 launch mecamind_bringup mapping.launch.py backend:=gazebo \
    auto_route:=true shutdown_on_route_complete:=true
# 5~6 分钟后自动结束。检查产物：
ls maps/ # mecamind_three_room_map.pgm/.yaml + 路线报告 JSON
cat maps/mecamind_three_room_route_report*.json | python3 -m json.tool
# 报告里 reached_waypoints 应为 9/9, auto_map_saved 应为 true
```

路线驱动器工作中的 RViz 画面（lite 后端，`with_rviz` 保持默认 `true` 即可旁观）：机器人正沿预设路点推进，图中的黄色细线是 TF 显示的父子坐标系连线（`map/odom` 原点指向机器人），地图随行进 逐块点亮：



半自动建图进行中：路线驱动器带着机器人自动探索

第 10 步：地图质量分析与资产管理

新房交付要请验房师：他不凭感觉说"看着挺好"，而是拿着标准 清单逐项量化——空鼓几处、阴角偏差几毫米，最后给出"可收房/ 需整改"的结论。地图也一样："看上去像个房子"不代表能拿去导航。 本步给地图请一位验房师：对照世界真值逐项打分，覆盖率多少、 四面墙实不实、自由区连不连通，量化到数字才能自动验收。

10.1 目标

写离线工具 `map_quality_analyzer.py`：输入 PGM/YAML 地图 + world 文件（真值），输出覆盖率、连通性、四面墙完整性等指标和一个结论（`navigation_ready` / `needs_remap` 等），让"图建得好不好"可量化。

10.2 指标设计

指标	算法	意义
coverage	ROI 内已知格 / （已知+未知）	探索是否覆盖全屋
unknown_ratio	ROI 内未知格占比	有没有大片没扫到的区域
largest_free_component	对空闲格做 BFS 洪水填充，取最大连通域占比	自由区是否连成一片（碎块=导航会失败）
wall_scores	按 world 真值沿四面墙采样，统计命中占据格的比例	每面墙画实了没有

ROI（分析范围）从 world 真值取，并**向内收缩**一点——SLAM 会把 地图裁到墙的内表面，收缩避免把墙外区域算成"未覆盖"。

10.3 踩坑实录：PGM 的 Y 轴方向

这个坑与 9.4 是同一约定的镜像：`map_saver` 写的 PGM，row 0 对应 `y_max`；而分析器最初按 `OccupancyGrid` 习惯（row 0 = `y_min`）读取，导致整张图上下颠倒——采样"北墙"实际采到了南墙的位置，墙体得分全错。修复：加载 PGM 后立即垂直翻转（`np.flipud`），统一回 `OccupancyGrid` 约定再做所有计算。

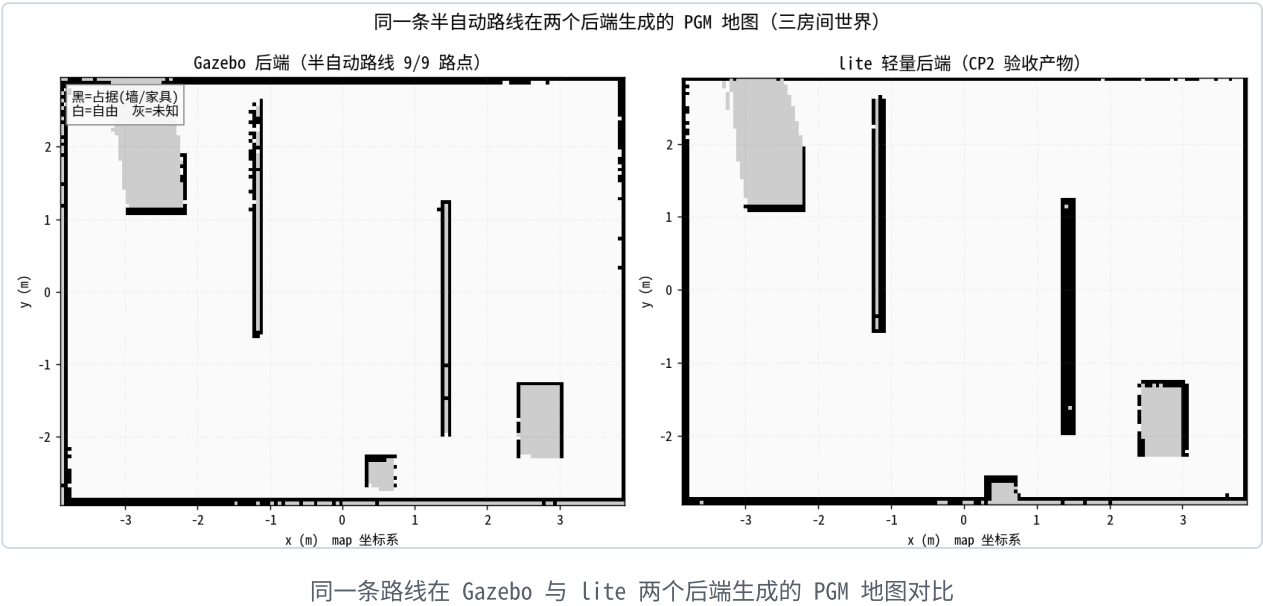
方法论：凡是涉及两种格式互转的代码，先写一个"已知答案"的小用例验证方向约定（比如在地图左下角画一个标记点，转换后检查 它还在不在左下角）。

10.4 配套工具与验证

`map_asset_manager.py` 给每张图登记档案（manifest JSON），按 质量分挑出最优地图。验证：

```
python3 -m mecamind_tools.map_quality_analyzer \
  maps/mecamind_three_room_map.yaml \
  --world src/mecamind_tools/worlds/three_room_house.world --json | python3 -m json.tool
# 期望：coverage ≈ 0.94+, missing_walls 为空, verdict 为 navigation_ready*
```

下图是同一条半自动路线在两个后端产出的 PGM 地图对比（黑=占据、白=自由、灰=未知）。两张图的房间结构、家具位置高度一致，这正是"统一接口 + 统一路线"想要的结果；差异主要在噪点分布上——Gazebo 有物理打滑所以墙面略毛糙，lite 后端则更干净：



第 11 步：CP2 自动验收脚本

工厂流水线的末端都有一道出厂质检：按下按钮，机器自动完成 装配、通电、测试、打合格证，任何一环不达标就亮红灯。有了 第 9 步的自动建图和第 10 步的质量打分，我们就能把整条链拼成 一键质检：改了任何代码，跑一遍脚本就知道建图功能还好不好—— 这比"我记得上周还能跑"可靠得多。

11.1 目标

写 `scripts/test_cp2.sh`：一条命令完成"无头半自动建图 → 存图 → 质量分析 → 判定通过/失败"，两个后端都能跑。这就是第二课的 自动验收 (Checkpoint 2)。

11.2 脚本结构

```
MECAMIND_CP2_BACKEND=${MECAMIND_CP2_BACKEND:-lite} # 环境变量选后端
# ① 锁目录防止并发运行：mkdir 是原子操作，天然适合当锁
mkdir /tmp/mecamind_cp2.lock.d || { echo "已有一套 CP2 在运行"; exit 1; }
trap 'rm -rf /tmp/mecamind_cp2.lock.d; kill $LAUNCH_PID 2>/dev/null' EXIT

# ② 无头启动建图 (with_rviz:=false use_gui:=false auto_route:=true)
# ③ 等待 launch 进程自己退出 (路线完成即自动 Shutdown, 见 9.2)
# ④ 跑质量分析，输出存文件再用 python 解析 (管道直接解析容易丢)
# ⑤ 综合判定：路点到达数 ≥ 2/3 且质量结论合格才算 PASS
```

要点：

- 用独立的 `ROS_DOMAIN_ID` 跑测试，避免与开发环境串台；
- Gazebo 后端的等待上限放宽 (RTF 低，墙钟 600s ≈ 仿真 300s)；
- 失败时保留日志与半成品地图，作为"失败证据"供排查。

11.3 验证

```
./scripts/test_cp2.sh # lite 后端，约 2 分钟
MECAMIND_CP2_BACKEND=gazebo ./scripts/test_cp2.sh # Gazebo 后端，约 8 分钟
# 输出末尾应有 [CP2] PASS 及质量指标汇总
```

实测基准 (供对照)：Gazebo 后端 9/9 路点、coverage 0.946、无缺失墙、verdict navigation_ready_best_effort。

第 12 步：搭建 Nav2 导航栈

自主导航像一支配合默契的车队：领航员摊开地图规划路线 (planner_server)，司机盯着路面打方向盘踩油门 (controller_server)，副驾负责确认"我们现在在地图哪里" (AMCL)，车陷住了还有人下车推 (behavior_server)，而车队 队长 (bt_navigator) 统筹所有人、对外只留一个电话。Nav2 不是 一个节点而是一整支车队——本步把它请进项目，并让它的每一脚 "油门"都过我们第 7 步的安全门。

12.1 目标

在第 9 步建好的地图上实现自主导航：RViz 里点一个目标，机器人自己 规划路径、避障开过去。一条命令 `ros2 launch mecamind_bringup navigation.launch.py backend:=gazebo|lite` 拉起全部。

12.2 原理 ①：代价地图——Nav2 的"危险度地图"

进入模块分工前，先理解贯穿 Nav2 的一个基础概念。代价地图（costmap）把环境离散成格子，每格一个 0~254 的"危险度"： 障碍本体是 254（致命），向外按指数衰减出一圈"膨胀带"， 让规划器天然倾向于离墙远一点的路径。本项目用两张：

	局部代价地图	全局代价地图
坐标系	odom（连续不跳变）	map（绝对准确）
范围	3x3m 滚动窗口跟着车走	整张已知地图
数据源	只有实时激光	静态地图 + 实时激光叠加
服务对象	DWB 局部避障	全局规划器

局部图用 odom 系是刻意的：控制回路每 0.1 秒执行一次，受不了 AMCL 修正带来的位姿跳变；而全局规划一次规划几秒钟才一次， 用 map 系拿绝对准确的环境。这又是第 6 步 TF 两段式设计的一次落地——你会发现整个导航体系都建立在那个设计之上。

12.3 原理 ②：Nav2 的模块分工（先建立整体认知）

Nav2 不是一个节点，是一组各司其职的 lifecycle 节点：

节点	职责	类比
map_server	把 PGM/YAML 地图发布为 /map	地图册
amcl	粒子滤波定位，输出 TF map→odom	"我在哪"
planner_server	全局规划（NavFn/Dijkstra 搜路径）	导航软件算路线
controller_server	局部控制（DWB 逐周期算速度）	方向盘和油门
behavior_server	卡住时的恢复行为（旋转/后退/等待）	自救动作
bt_navigator	行为树总指挥，对外提供 NavigateToPose action	大脑
velocity_smoother	速度平滑（限加速度）	防急刹
local/global_costmap	代价地图（障碍膨胀）	危险度地图

AMCL 与 SLAM 的关系：建图阶段 slam_toolbox 同时干定位+建图； 导航阶段地图已知，换成更轻量的 AMCL 只做定位。两者输出同一个 TF map→odom，下游无感。

12.4 操作 ①：准备地图资产

把 CP2 产出的最优地图收编为包内资产，随包安装：

```
cp maps/mecamind_cp2_gazebo_map.pgm src/mecamind_tools/maps/mecamind_three_room_nav.pgm
cp maps/mecamind_cp2_gazebo_map.yaml src/mecamind_tools/maps/mecamind_three_room_nav.yaml
# 记得改 yaml 里的 image: 字段指向新文件名，然后 colcon build
```

12.5 操作 ②：Nav2 参数中与"麦轮"强相关的三处

参数文件 `src/mecamind_tools/config/mecamind_nav2_params.yaml` 每项都有注释，这里强调专属于全向底盘的三处：

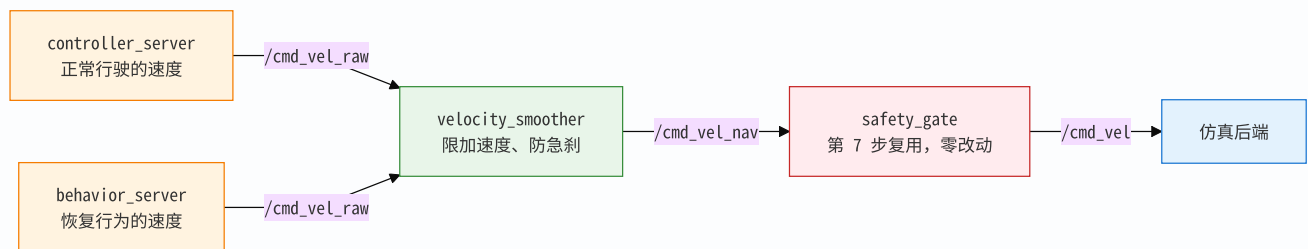
```
amcl:
  robot_model_type: nav2_amcl::OmniMotionModel  # ① 全向运动模型（差速模型没有 vy）

controller_server:
  FollowPath:
    min_vel_y: -0.25      # ② 允许横移：DWB 会在 vy 维度采样
    max_vel_y: 0.25
    vy_samples: 6

velocity_smoother:
  max_velocity: [0.35, 0.25, 1.2]  # ③ 三分量 [x, y, theta] 都要给 y 留空间
```

12.6 操作 ③：速度链路的重映射拼接（本步最核心的工程设计）

Nav2 默认把速度直接发 `cmd_vel`，会绕过我们的安全门。解决方案 全靠 launch 里的 remappings 拼接，不改任何 Nav2 代码：

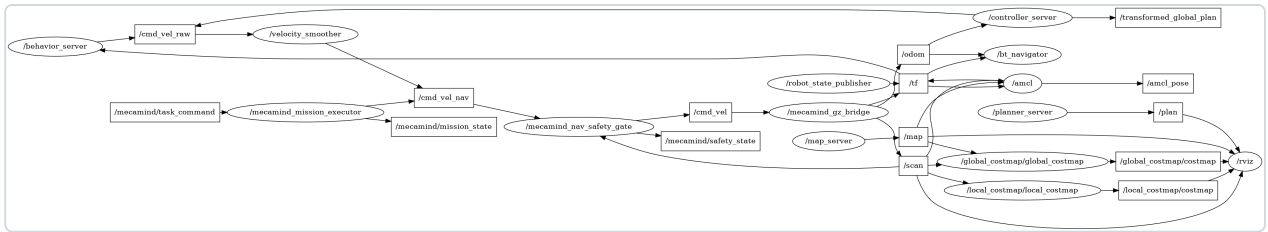


对应 `src/mecamind_bringup/launch/navigation.launch.py`：

```
controller_server = Node(..., remappings=[("cmd_vel", "/cmd_vel_raw")])
behavior_server   = Node(..., remappings=[("cmd_vel", "/cmd_vel_raw")])
velocity_smoother = Node(..., remappings=[
    ("cmd_vel", "/cmd_vel_raw"),          # 输入
    ("cmd_vel_smoothed", "/cmd_vel_nav"), # 输出 → 安全门入口
])
```

注意 `behavior_server` 也要重映射——恢复行为（旋转、倒车）同样会发速度，漏了它就等于给安全门开了个后门。

导航系统跑起来后的实际节点图印证了这条链路：图左上 `controller_server / behavior_server` → `/cmd_vel_raw` → `velocity_smoother` → `/cmd_vel_nav` → `mecamind_nav_safety_gate` → `/cmd_vel` → 仿真后端，一环不漏；图右侧则是 `map_server`、`AMCL`、代价地图等定位与规划部分：



导航系统运行时的节点与话题连接图：速度三级链路清晰可见

12.7 操作 ④：生命周期激活与初始位姿

Nav2 全家都是 lifecycle 节点，复用第 8 步的 `lifecycle_activator`，按 `map_server` → `amcl` → 其余 的顺序 `configure`→`activate`（地图先有，定位才能配置，其余才能启动）。

AMCL 还需要初始位姿才开始工作。因为出生点统一在原点（第 6 步），激活器最后向 `/initialpose` 发布 (0,0,0) 即可，参数 `set_initial_pose: false` 表示不用 AMCL 自己的参数写死。Gazebo 冷启动慢，激活延迟 8 秒，服务等待上限 60 秒。

12.8 踩坑实录：导航又慢又卡，最后超时

Gazebo 后端首测：到第一个目标正常，第二个目标反复卡住直到超时。三个原因叠加，逐一修复：

1. **CPU 饱和拖低 RTF**：controller 20Hz × DWB 采样 $18 \times 10 \times 24 = 4320$ 条轨迹/周期，WSL2 扛不住 → 降到 10Hz、 $10 \times 6 \times 16 = 960$ 条；
2. **路径贴墙**：`inflation_radius` 0.32m 太小，规划的路径紧贴障碍，车频繁进入安全门减速区越走越慢 → 加大到 0.45m，路径自动离墙远；
3. **超时按墙钟算**：RTF≈0.5 时 240s 墙钟只等于 120s 仿真 → 任务层超时用仿真时间 240s，外层脚本墙钟上限放宽到 420s。

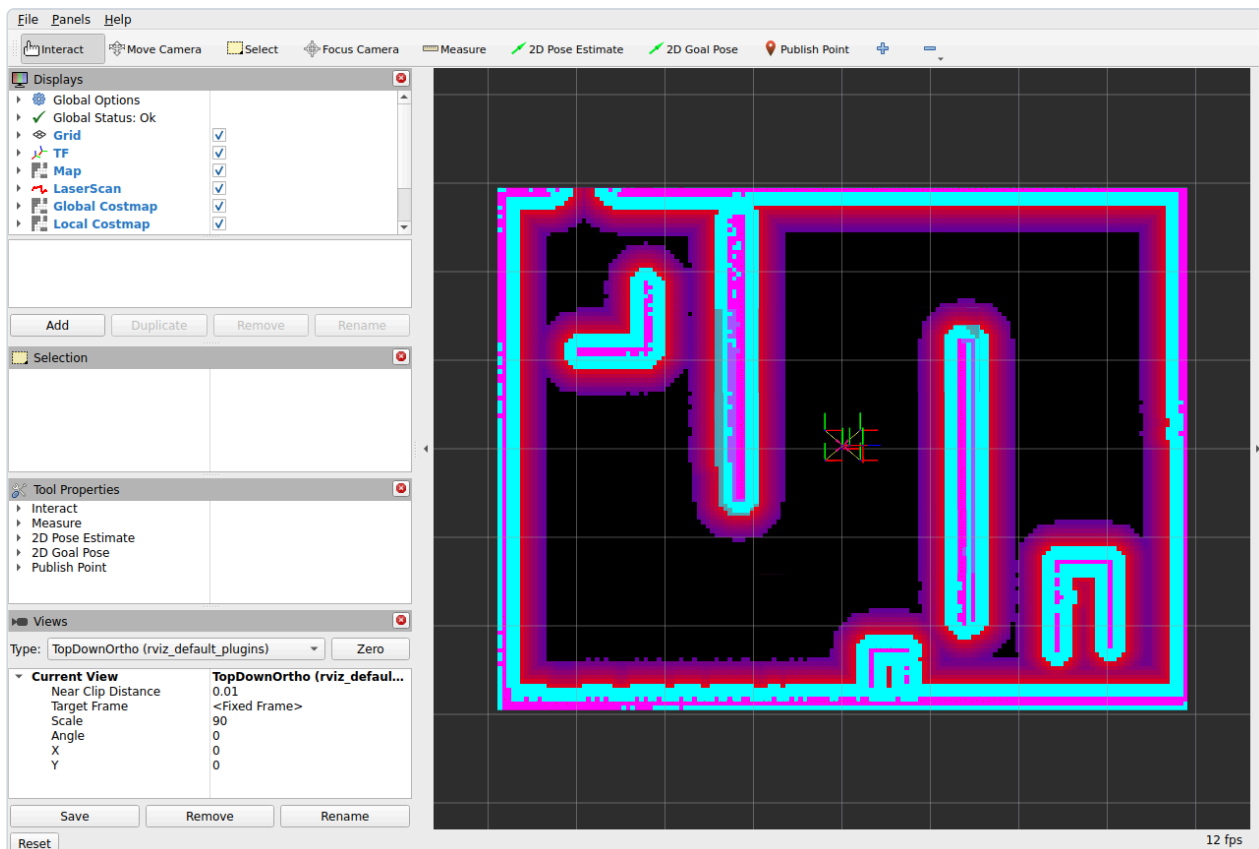
方法论：仿真里的"性能问题"先查 RTF (`gz stats`)。RTF 低时，一切按墙钟设定的预期都要打对折，或者干脆全部改用仿真时间。

12.9 验证

```
bash scripts/cleanup_ros2.sh --all
ros2 launch mecamind_bringup navigation.launch.py backend:=gazebo

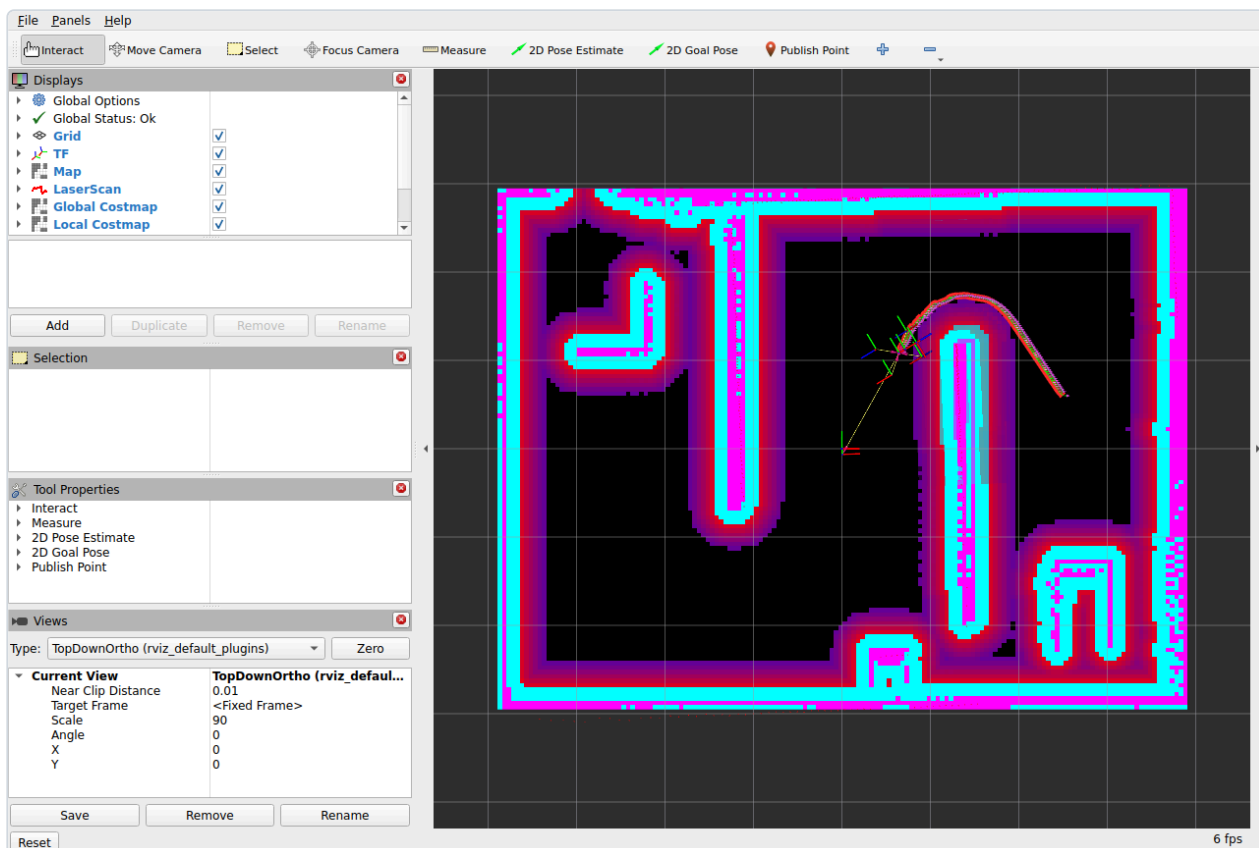
# RViz 检查清单：地图显示 → 代价地图彩色膨胀带 → 机器人在原点、
# 周围有绿色粒子云 (AMCL) → 工具栏 Nav2 Goal 点个目标 → 红色路径出现，车开动
```

启动完成后的 RViz 画面：静态地图之上叠加了代价地图，青色是 致命区（障碍本体+机器人半径），紫红色渐变是膨胀衰减带——规划器会自动避开这些高代价区域。机器人在原点，TF 坐标轴清晰：



导航系统启动完成：代价地图的青色致命区与紫红色膨胀带

发出目标后，全局规划器画出一条红色路径，注意它始终沿着代价 低谷走、与青色致命区保持距离（这就是 12.8 里加大 `inflation_radius` 换来的效果），机器人随即沿路径开动：



导航进行中：红色全局路径绕开高代价区通往目标点

12.10 动手练习

1. 观察 AMCL 粒子云收敛：刚启动时粒子散布较大，让车开动几米后 粒子迅速聚拢——运动+观测才能消除位姿的不确定性；
2. 把 `inflation_radius` 临时改回 0.32 重跑，在 RViz 里对比全局 路径与墙的间距，体会 12.8 修复的效果；
3. 用 `ros2 topic hz /cmd_vel_raw /cmd_vel_nav /cmd_vel` 同时观察 三级速度话题，验证 12.6 的链路每一环都在流动；
4. 思考题：为什么局部代价地图的 `global_frame` 是 `odom` 而全局的 是 `map`？如果把局部的也改成 `map` 会发生什么？（答案在 12.2）

第 13 步：任务层——命名目标与任务执行器

去公司拜访，你只需要在前台说"我找张经理"：前台查通讯录知道 张经理在 508，带你过去，中途还会告诉你"到了/他不在"。你不需要 知道 508 在几层几号楼——这正是任务层的角色。Nav2 只认坐标和 四元数（相当于"508 房间的经纬度"），任务层把"去卧室"这样的人话翻译成坐标，并把执行进度用人话播报回来。

13.1 目标

让上层（语音、调度、脚本）用一句 JSON 就能指挥机器人：

```
ros2 topic pub --once /mecamind/task_command std_msgs/msg/String \
  '{data: "{\"intent\": \"navigate\", \"target\": \"bedroom\"}'}'
```

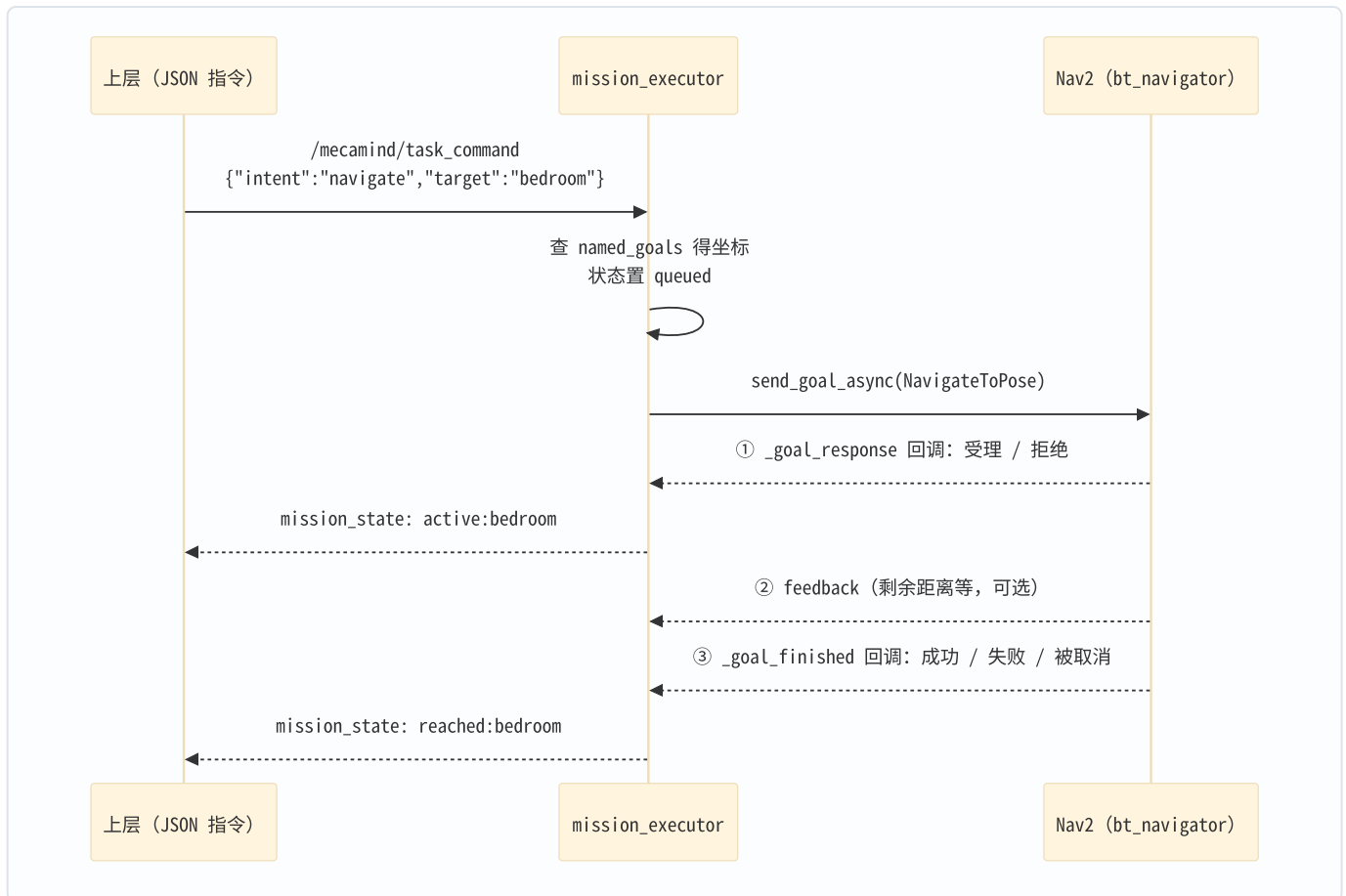
13.2 原理 ①：为什么要在 Nav2 上再包一层

Nav2 的接口是 `action NavigateToPose`，要填坐标、四元数，对上层 太底层了。任务层做三件事：

1. 语义化： `bedroom` → 查 `mecamind_named_goals.yaml` 得到坐标；
2. 协议简化： `topic + JSON 字符串`，任何语言/工具都能发；
3. 状态外发： `/mecamind/mission_state` 发布 JSON 状态（`queued/active/reached/failed...`），上层可监控。

13.3 原理 ②：action 的三层回调链（ROS 2 重点知识）

`topic` 是"发了就完"，`action` 是"有受理、有反馈、有结果"的长任务协议。 `mission_executor.py` 与 Nav2 的完整交互时序：



配套两个工程细节值得学习：

- **目标序列号** (`_goal_serial`)：每个新任务分配递增序号，旧任务迟到的回调发现序号对不上就直接丢弃——防止“上一个目标的失败回调污染当前任务状态”；
- **两阶段发送**：收到指令先置 `queued`，由定时器等 Nav2 action 服务就绪后再真正发送，启动竞态就不存在了。

`intent` 支持 `navigate` (单点)、`patrol` (按 `waypoints` 文件巡航)、`stop` (取消当前目标并刹车)。

代码精讲 (`mission_executor.py` 的指令入口 `_task_cb`)：

```

def _task_cb(self, msg: String) -> None:
    try:
        data = json.loads(msg.data)
    except Exception as exc:
        self.get_logger().warn(f"Invalid task_command JSON: {exc}")
        return
    # ① 外部输入永远不可信：解析失败只告警不崩溃
    intent = str(data.get("intent", "unknown")).strip().lower() # ② 容忍大小写
    target = str(data.get("target", "")).strip()
    requires_confirmation = bool(data.get("requires_confirmation", False))

    # ③ 需要人确认的动作绝不自动执行；但 stop/cancel 例外——停止永远无条件
    if requires_confirmation and intent not in {"stop", "cancel"}:
        self.set_state("idle", "awaiting_confirmation", target)
        return
    # ④ 之后按 intent 分发到 _enter_navigate / _enter_patrol / _stop_all
  
```

这段代码浓缩了处理外部指令的三条纪律：**永不信任输入**（try 包住 解析）、**归一化再比较**（strip + lower）、**安全指令拥有最高 优先级**（stop 绕过一切拦截）。写任何“接收指令”的节点都可以套用。

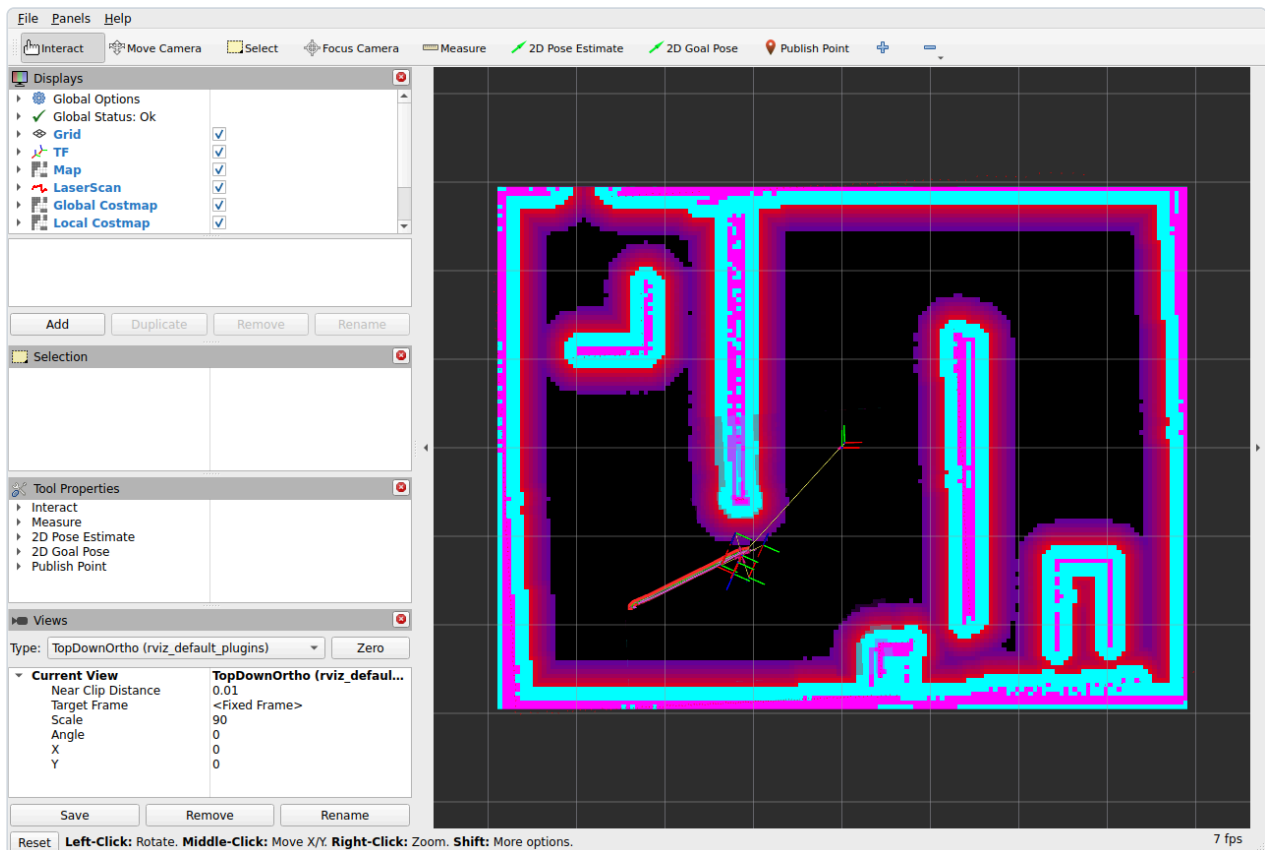
13.4 验证

```
# 前提：第 12 步的导航系统在跑
ros2 topic echo /mecamind/mission_state &    # 盯着状态流

ros2 topic pub --once /mecamind/task_command std_msgs/msg/String \
  '{data: "{\\"intent\\": \\"navigate\\", \\"target\\": \\"living_room\\""}'}'
# 状态应依次出现 queued → sending → active:living_room → reached:living_room

ros2 topic pub --once /mecamind/task_command std_msgs/msg/String \
  '{data: "{\\"intent\\": \\"patrol\\""}'}'          # 三点巡航
ros2 topic pub --once /mecamind/task_command std_msgs/msg/String \
  '{data: "{\\"intent\\": \\"stop\\""}'}'           # 中途取消，车应立即停
```

一条 JSON 指令发出后，机器人已在赶往目标的路上——注意这次 没有人碰 RViz 的 Nav2 Goal 按钮，路径完全由任务层触发：



JSON 巡航指令生效：机器人沿红色路径驶向路点

13.5 动手练习

1. 在 `mecamind_named_goals.yaml` 里加一个新目标 `kitchen_corner`（坐标自己在 RViz 里用 Publish Point 工具点出来），rebuild 后用 JSON 指令导航过去；
2. 发一条故意写错的 JSON（比如 `{"intent": "navigate"` 少个括号），确认执行器只告警不崩溃——对照 13.3 代码精讲的第①条纪律；

3. 巡航进行到一半发 stop，再立刻发一个 navigate——观察 mission_state 的状态切换是否干净利落（这背后是 13.3 的目标序列号机制在防止旧回调污染）。

第 14 步：CP3 验收与里程计漂移观察

闭上眼睛沿直线走 20 米，睁眼时你多半已经偏出一两米——每一步的小误差在不断累加，而你毫无察觉。轮式里程计就是“闭眼走路”：只靠轮子转了多少来推算位置，麦轮还特别爱打滑，误差累积更快。解决办法和人一样：时不时“睁眼看一下路标”（激光扫描对比地图），把累积的偏差一次性修正掉。本步先给导航系统做完整验收，再亲眼观察这个“睁眼修正”的过程。

14.1 CP3 自动验收

mecamind_tools/nav_acceptance.py + scripts/test_cp3.sh 组成第三课的自动验收：无头启动导航栈，依次发 3 个命名目标 + 巡航 + 取消，全部按预期完成才 PASS。设计要点：

- **黑盒测试**：只通过 /mecamind/task_command 和 /mecamind/mission_state 交互，与真实用户完全同路径；
- **先等定位再发目标**：等 /amcl_pose 出现后再静置 3 秒（settle_sec），否则 TF 缓冲不足会报 extrapolation 错误；
- **每个目标允许重试一次**，且重试时先等执行器**受理新任务**（出现 queued/sending/active）再判终态，避免拿旧状态误判；
- **失败时收集证据**：最后的 AMCL 位姿、安全门状态、mission_state 尾巴。

```
./scripts/test_cp3.sh # lite, 约 3 分钟
MECAMIND_CP3_BACKEND=gazebo ./scripts/test_cp3.sh # Gazebo, 约 10 分钟
```

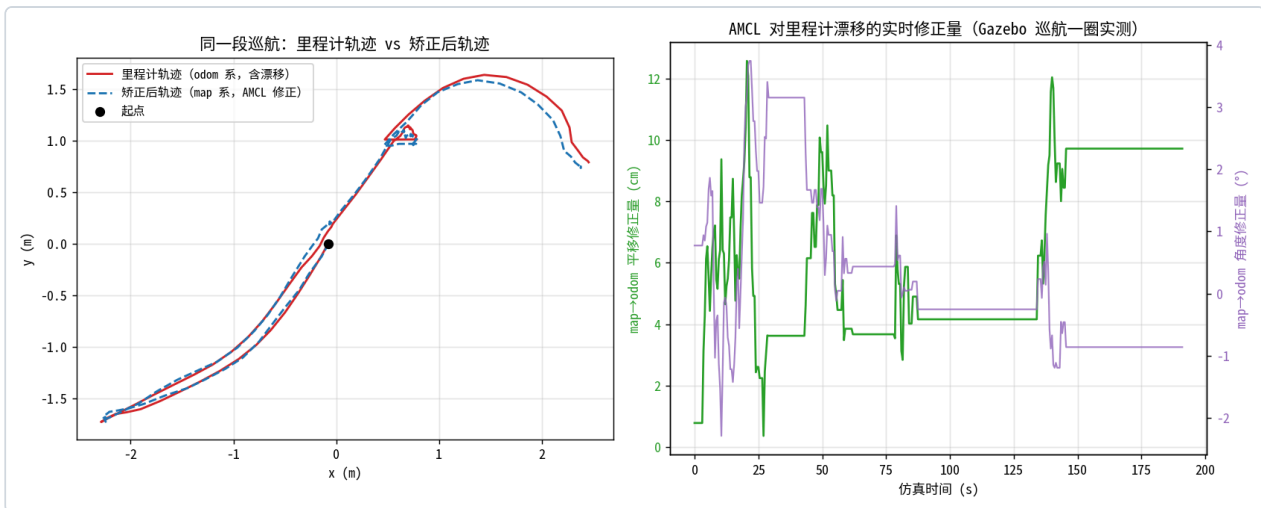
14.2 亲眼看见漂移矫正

Gazebo 的麦轮有真实打滑，/odom 会漂移，AMCL 在持续修正。用两个终端就能“看见”这件事：

```
# map→odom: AMCL 的修正量。车跑起来后数值阶梯状跳变 = 每次滤波更新
ros2 run tf2_ros tf2_echo map odom

# 录制并画图（本仓库自带工具）
python3 scripts/record_drift.py /tmp/drift.csv &
# ...让车巡航一圈...
# 出图：里程计轨迹 vs 修正后轨迹 + 修正量时间序列
```

下面是在 Gazebo 后端巡航一圈的实测数据。左图：红色实线是纯里程计轨迹（含漂移），蓝色虚线是 AMCL 矫正后的轨迹，二者随行驶逐渐分开——分开的量就是累计漂移。右图：绿色是 map→odom 的平移修正量（厘米级、持续变化），紫色是角度修正量，都呈明显的阶梯状：



Gazebo 巡航一圈实测：里程计轨迹 vs 矫正后轨迹，及 AMCL 修正量时间序列

修正为什么是阶梯状？AMCL 的 `update_min_d/update_min_a` (0.15) 规定车挪够 15cm/转够 0.15rad 才做一次滤波更新——精度换 CPU 的取舍。

14.3 矫正原理概要

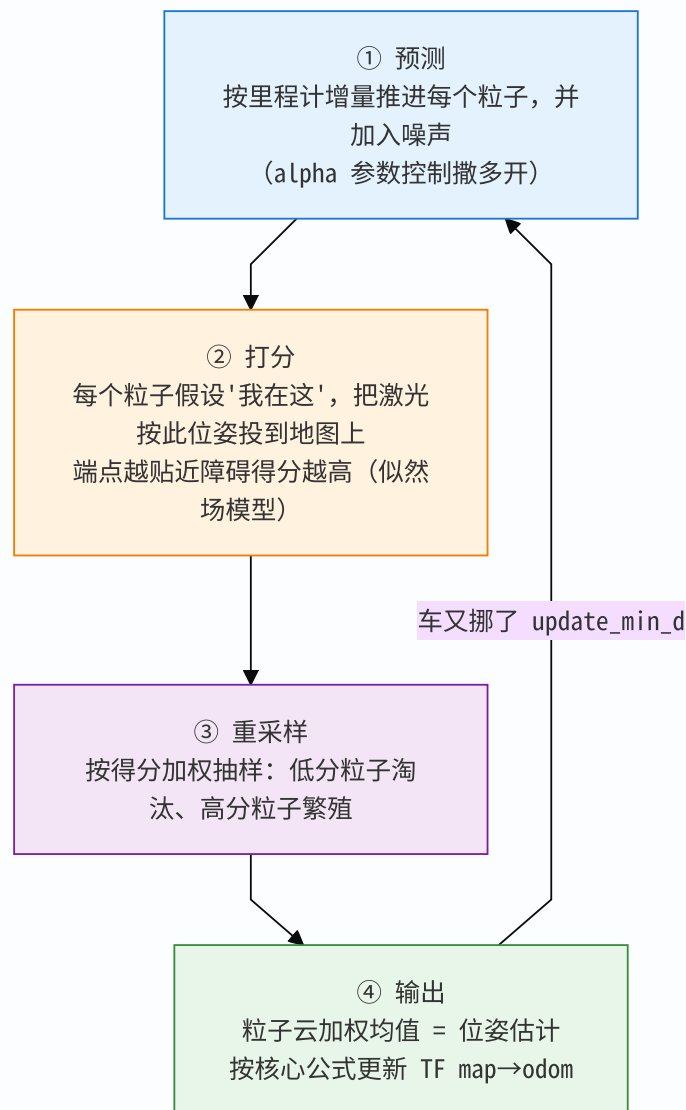
完整推导见 `docs/10.3.3_专题_里程计漂移与矫正.md`，这里给出 核心结论：

核心公式。AMCL 通过激光匹配估计出机器人在地图中的真实位姿 $T_{map \rightarrow base}$ ，而里程计给出 $T_{odom \rightarrow base}$ 。二者的"差"就是应发布的修正量：

$$T_{map \rightarrow odom} = T_{map \rightarrow base} \cdot (T_{odom \rightarrow base})^{-1}$$

直觉理解：把 odom 坐标系整体"搬动"一下，使得 odom 系里的机器人 恰好落到 map 系里激光认定的位置上。里程计本身一个字不改， 所有修正都体现在这个"搬动量"里。

AMCL 怎么得到 $T_{map \rightarrow base}$ ——粒子滤波四步循环：



用一句话总结整个定位体系：里程计管连续，激光管正确，map→odom 是两者的差额账本。

14.4 动手练习

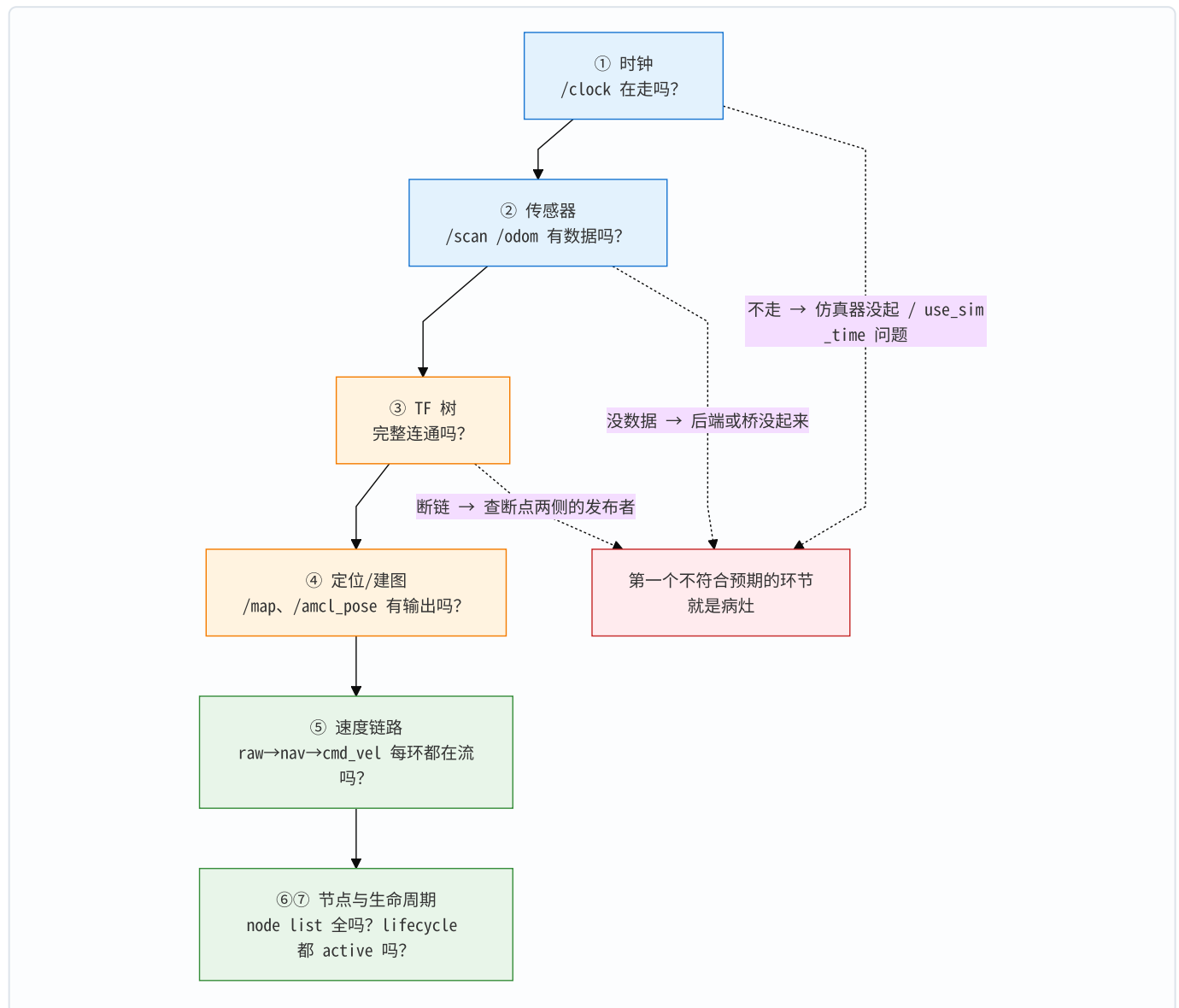
1. 分别在 lite（完美里程计）和 Gazebo（真实打滑）后端跑 `tf2_echo map odom`，对比修正量的大小和变化频率；
2. 给 lite 后端注入 `odom_xy_noise_std:=0.02` 的人工漂移，观察 AMCL 把它"接管"——修正量开始阶梯状增长；
3. 把 AMCL 的 `update_min_d` 改成 0.5（挪半米才更新），观察修正跳变变大变稀疏——理解这对贴墙行驶的风险。

第 15 步：调试方法与避坑手册

好医生看病不靠猜：先量体温血压（生命体征），再听诊化验，一步步 缩小范围，最后才下诊断。机器人系统"生病"时也一样——时钟就是 体温，传感器数据就是血压，TF 树就是化验单。本步教你按固定顺序 "问诊"，并附上本项目实战期间积累的完整"病历库"。

15.1 通用排查流程：从数据流上游往下游查

系统不工作时，别猜，按数据流顺序逐环验证，坏的一定是第一个 不符合预期的环节：



每一步对应的命令：

- | | |
|---------------------|---|
| ① /clock 在走吗? | <code>ros2 topic echo /clock --once</code> |
| ② /scan /odom 有数据吗? | <code>ros2 topic hz /scan</code> |
| ③ TF 树完整吗? | <code>ros2 run tf2_tools view_frames</code> |
| ④ SLAM/AMCL 输出了吗? | <code>ros2 topic hz /map / ros2 topic echo /amcl_pose --once</code> |
| ⑤ 速度链路通吗? | 依次 <code>echo /cmd_vel_raw -> /cmd_vel_nav -> /cmd_vel</code> |
| ⑥ 节点都活着吗? | <code>ros2 node list</code> |
| ⑦ lifecycle 激活了吗? | <code>ros2 lifecycle get /map_server</code> |

15.2 高频故障速查表（本项目实战全记录）

现象	根因	解法
RViz/Gazebo 打不开，报 display 错误	DISPLAY 被设成 Windows IP	<code>export DISPLAY=:0</code>
TF_OLD_DATA 刷屏	上次的 gz sim 没死透，旧 /clock 污染	<code>bash scripts/cleanup_ros2.sh --all</code> 再启动
节点启动但不干活	lifecycle 节点没被激活	查 <code>ros2 lifecycle get</code> ，确认激活器在跑
SLAM 报 TF 外推错误、丢激光	某个节点忘了 <code>use_sim_time:=true</code>	全员检查，包括 RViz 和命令行工具
地图缺墙	激光远处命中密度不足 (<code>min_pass_through</code>)	见 8.5
map_saver 订阅失败	sim time 下临时节点等不到锁存消息	带 <code>-p use_sim_time:=true</code> ；或像 9.4 直接写文件
路线驱动器总报 no progress	用墙钟判断超时而 <code>RTF < 1</code>	改用 <code>node.get_clock()</code> （见 9.3）
导航慢、反复卡住	CPU 饱和 + 路径贴墙 + 超时按墙钟	见 12.8 的三项修复
起步就报 Initial robot pose is not available	AMCL 刚发位姿 TF 缓冲不足	等 <code>/amcl_pose</code> 后静置几秒再发目标
窗口撑满屏幕拖不动（WSLg）	WSLg 窗口管理怪癖	<code>Win+↓</code> 还原；或 <code>python3 scripts/resize_window.py</code>
pytest/构建正常但 <code>ros2 run</code> 找不到节点	<code>entry_points</code> 没登记或没重新 build	改 <code>setup.py</code> + <code>colcon build</code>
两人同网互相干扰	<code>ROS_DOMAIN_ID</code> 相同	各自设不同值

15.3 版本管理工作流（课程标签怎么打）

本项目用 git 标签固化每节课的成品，你随时能取到可运行的参考版本：

```
# 取某节课的成品对照自己的实现
git fetch --tags
git tag -l                # 查看有哪些标签
git checkout lesson2.0    # 处于"游离 HEAD"状态，只读参考
git checkout main         # 看完回主线

# 自己练习时也建议养成打标签的习惯：一个阶段完成并验证通过后
git add -A && git commit -m "第二步完成：SLAM 建图跑通"
git tag my-step2 && git push origin my-step2

# 后续有小修补 → 打新的小版本号，不要移动已推送的标签
```

两条纪律：已推送的标签永远不要 `-f 移动`（别人本地缓存的 旧标签不会自动更新，会造成"同名不同码"的混乱）；标签打在 验证通过的提交上，保证任何时候检出的版本一定能跑。

15.4 三条最值钱的经验

1. **时间是仿真系统的第一公民。**90% 的诡异问题最后都能追到 时钟：use_sim_time 漏设、墙钟当仿真时间用、旧进程污染 /clock。写代码时凡涉及时间一律 `node.get_clock()`，凡启动节点一律带 `use_sim_time`，凡重启一律先清理进程。
 2. **接口契约先行。**先定死 topic 名、TF 树、出生点（第 6 步），再往两边填实现，双后端、多模块并行开发才不会互相踩。
 3. **每一步都留自动验收。**CP2/CP3 脚本让每次改动都能一条命令 回归，教学项目也值得按工程项目的标准来。
-

附录 A：命令速查表

```
# —— 构建 ——
cd ~/mecamind_ros2
colcon build # 全量
colcon build --packages-select mecamind_tools # 单包
source install/setup.bash # 每次 build 后必做

# —— 清理（每次启动前） ——
bash scripts/cleanup_ros2.sh --all

# —— 第一课：仿真 ——
ros2 launch mecamind_bringup gazebo.launch.py # Gazebo 后端
ros2 launch mecamind_bringup lite_sim.launch.py # 轻量后端

# —— 第二课：建图 ——
ros2 launch mecamind_bringup mapping.launch.py backend:=gazebo # 手动
ros2 launch mecamind_bringup mapping.launch.py backend:=lite \
  auto_route:=true shutdown_on_route_complete:=true # 半自动
ros2 run nav2_map_server map_saver_cli -f maps/mymap \
  --ros-args -p use_sim_time:=true # 存图
./scripts/test_cp2.sh # CP2 验收

# —— 第三课：导航 ——
ros2 launch mecamind_bringup navigation.launch.py backend:=gazebo
ros2 topic pub --once /mecamind/task_command std_msgs/msg/String \
  '{data: "{\\"intent\\": \\"navigate\\", \\"target\\": \\"bedroom\\"}"}'
ros2 topic pub --once /mecamind/task_command std_msgs/msg/String \
  '{data: "{\\"intent\\": \\"patrol\\"}"}'
ros2 topic pub --once /mecamind/task_command std_msgs/msg/String \
  '{data: "{\\"intent\\": \\"stop\\"}"}'
ros2 topic echo /mecamind/mission_state # 看状态
./scripts/test_cp3.sh # CP3 验收

# —— 诊断 ——
ros2 topic list / hz / echo / info -v
ros2 node list
ros2 run tf2_tools view_frames
ros2 run tf2_ros tf2_echo map odom # 看漂移修正量
ros2 lifecycle get /map_server
gz stats # 看 Gazebo RTF

# —— 课程标签 ——
git fetch --tags && git tag -l
git checkout lesson1.0 # 第一课成品; lesson2.0 = 第二课成品
```

附录 B：项目文件地图

mecamind_ros2/	
├── src/	
│ ├── mecamind_description/	
│ │ └── urdf/mecamind_mecanum.urdf.xacro	# 第 2 步：机器人骨架（ROS 侧）
│ ├── mecamind_gazebo/	
│ │ ├── models/mecamind_mecanum/model.sdf	# 第 3 步：物理模型（麦轮摩擦、传感器、驱动）
│ │ ├── worlds/three_room_house.sdf	# 第 3 步：Gazebo 三室户世界
│ │ ├── config/bridge.yaml	# 第 4 步：话题桥接映射表
│ │ └── launch/gazebo_sim.launch.py	# 第 4 步：Gazebo 后端启动
│ ├── mecamind_bringup/	# 顶层入口（只需记住这个包）
│ │ └── launch/	
│ │ ├── gazebo.launch.py	# 第 4 步入口
│ │ ├── lite_sim.launch.py	# 第 5 步入口
│ │ ├── mapping.launch.py	# 第 8~9 步入口
│ │ └── navigation.launch.py	# 第 12 步入口
│ └── mecamind_tools/	
│ ├── mecamind_tools/	
│ │ ├── world_geometry.py	# 第 5 步：世界几何解析
│ │ ├── sim_core.py	# 第 5 步：运动积分/激光求交
│ │ ├── simulator_node.py	# 第 5 步：轻量仿真器节点
│ │ ├── safety_layer.py	# 第 7 步：安全门
│ │ ├── lifecycle_activator.py	# 第 8/12 步：生命周期激活器
│ │ ├── mapping_route_driver.py	# 第 9 步：半自动建图司机
│ │ ├── map_quality_analyzer.py	# 第 10 步：地图质量分析
│ │ ├── map_asset_manager.py	# 第 10 步：地图资产登记
│ │ ├── mission_executor.py	# 第 13 步：任务执行器
│ │ ├── nav_acceptance.py	# 第 14 步：CP3 验收节点
│ │ └── auto_explore.py 等	# 进阶：全自动探索
│ ├── config/	
│ │ ├── mecamind_slam_toolbox_params.yaml	# 第 8 步
│ │ ├── mecamind_mapping_route.yaml	# 第 9 步
│ │ ├── mecamind_nav2_params.yaml	# 第 12 步
│ │ ├── mecamind_named_goals.yaml	# 第 13 步
│ │ └── mecamind_patrol_waypoints.yaml	# 第 13 步
│ ├── maps/mecamind_three_room_nav.*	# 第 12 步：默认导航地图
│ ├── rviz/*.rviz	# 各课 RViz 预置布局
│ └── worlds/three_room_house.world	# 第 5 步：轻量仿真世界
├── scripts/	
│ ├── cleanup_ros2.sh	# 进程清理（每次启动前）
│ ├── test_cp2.sh	# 第 11 步：建图验收
│ ├── test_cp3.sh	# 第 14 步：导航验收
│ ├── record_drift.py	# 第 14 步：漂移记录
│ └── resize_window.py	# WSLg 窗口调整
├── maps/	# 运行产物：地图与报告（不提交）
└── docs/	# 课程讲义与本教程

附录 C：里程碑自查表

每完成一步，勾掉对应的检查项再前进。任何一项不通过， 回到该步的"常见错误"排查：

第 0~1 步（环境与骨架）

- ☐ `echo $ROS_DISTRO` 输出 `jazzy`, `gz sim shapes.sdf` 能开窗口
- ☐ `ros2 pkg list | grep mecamind` 列出四个包
- ☐ 理解：改代码后必须 `build + source` 才生效

第 2~4 步（建模与 Gazebo 后端）

- ☐ `check_urdf` 通过, RViz 能显示完整机器人
- ☐ Gazebo 里机器人能被 `/cmd_vel` 驱动, 能横移
- ☐ `ros2 topic hz /scan` $\approx 10\text{Hz}$, `view_frames` 的 TF 树连到 `lidar_link`
- ☐ 理解：URDF 管 TF/显示, SDF 管物理; 桥接方向为什么是单向

第 5~7 步（轻量后端与安全门）

- ☐ `lite` 后端下同样的 topic/TF 检查全部通过
- ☐ 同一套 `teleop` 命令在两个后端都能开车（接口契约生效）
- ☐ 朝墙发全速指令, `/cmd_vel` 被安全门衰减到 0
- ☐ 理解：本体速度 $\rightarrow$ 世界位移的旋转变换公式

第 8~11 步（建图）

- ☐ 手动遥控能建出完整地图并成功存图（带 `use_sim_time`）
- ☐ `auto_route:=true` 半自动建图 9/9 路点, 自动存图退出
- ☐ 质量分析 `coverage > 0.9`、无缺失墙
- ☐ `./scripts/test_cp2.sh` 输出 `PASS`
- ☐ 理解：SLAM 的输入是 TF 而非 `/odom`; `min_pass_through` 的坑

第 12~14 步（导航）

- ☐ RViz 点目标能导航到达, 路径离墙有余量
- ☐ JSON 指令 `navigate/patrol/stop` 全部生效, `mission_state` 正确
- ☐ `./scripts/test_cp3.sh` 输出 `PASS`
- ☐ `tf2_echo map odom` 能看到 Gazebo 后端的漂移修正
- ☐ 理解：速度链路四级拼接; 局部/全局代价地图为何用不同坐标系

附录 D：如何把本项目改造成你自己的机器人

骨架是通用的, 换皮只需要动这些地方（按工作量从小到大）：

① 换环境（半天）

- 改两份世界文件（`three_room_house.sdf` 和 `.world`），保持两边 几何一致；
- 重新跑第 9 步建图（路线文件 `mecamind_mapping_route.yaml` 按新 户型重写路点）；
- 更新 `mecamind_named_goals.yaml` / `mecamind_patrol_waypoints.yaml` 的坐标。

② 换传感器（半天~一天）

- 雷达线数/量程：改 SDF 传感器段 + lite 仿真器参数 + `max_laser_range` (SLAM) / `laser_max_range` (AMCL)；
- 注意第 8.5 步的教训：线数变少后 `min_pass_through` 和噪声 配置要重新评估。

③ 换底盘（一到三天）

- 差速底盘：URDF/SDF 改成两轮+万向轮，驱动插件换 `gz-sim-diff-drive-system`；lite 仿真器把 `vy` 强制为 0；Nav2 侧 `OmniMotionModel` 换回 `DifferentialMotionModel`，DWB 的 `min_vel_y/max_vel_y/vy_samples` 全部归零；
- 尺寸变化：同步改 URDF/SDF 几何、costmap 的 `footprint`、安全门阈值、`robot_radius` (lite 仿真器)。

④ 上真机（另一个故事）

- 保住接口契约就能无缝切换：驱动板节点提供 `/odom` + TF `odom→base_footprint` + 订阅 `/cmd_vel`，雷达驱动发 `/scan`，全体 `use_sim_time` 改回 `false`；
- `mecamind_tools/launch/mecamind_real.launch.py` 预留了这个 形态的入口。

改造时的检验标准始终不变：第 6 步的接口自查 + CP2/CP3 脚本（阈值按新环境微调）。只要验收脚本还能跑通，你的改造就是安全的。

附录 E：术语表

按教程出现顺序排列，供随时回查：

术语	一句话解释	详见
WSL2 / WSLg	Windows 内置的 Linux 虚拟化及其 GUI 支持	第 0 步
colcon	ROS 2 的构建工具, 编译工作区里所有包	第 1 步
package (包)	ROS 2 代码的发布单元, 含 package.xml	第 1 步
Node (节点)	独立运行的功能单元 (进程)	1.5
Topic (话题)	多对多异步广播的通信方式	1.5
Service (服务)	一问一答式远程调用	1.5
Action (动作)	有受理/反馈/可取消的长任务协议	1.5、13.3
TF	分布式坐标变换系统	1.5、6.3
URDF / xacro	ROS 侧机器人描述格式及其宏语言	第 2 步
base_footprint	机器人在地面投影的虚拟根坐标系	2.2
SDF	Gazebo 侧的机器人/世界描述格式	第 3 步
各向异性摩擦 (fdir1/mu2)	用摩擦方向数学等价麦轮小辊子	3.3
RTF (实时因子)	仿真时间流速 / 真实时间流速	4.4、12.8
use_sim_time	让节点时钟跟随 /clock 的关键参数	4.4
ros_gz_bridge	Gazebo 与 ROS 2 的话题翻译器	第 4 步
remapping (重映射)	不改代码改话题接线的机制	5.8、12.6
slab 法	射线与矩形求交的经典算法	5.5
C-space 膨胀	障碍膨胀机器人半径、机器人当质点	5.7、12.2
QoS	话题的服务质量合同 (不匹配收不到消息)	6.4
TRANSIENT_LOCAL	锁存 QoS: 新订阅者可补收最后一帧	6.4
SLAM	同时定位与建图	第 8 步
扫描匹配	拿激光对地图搜索最佳位姿	8.2
回环检测	识别"回到走过的地方"以消除累积误差	8.3
lifecycle 节点	有 configure/activate 状态机的节点	8.4
OccupancyGrid	占据栅格地图消息 (-1 未知/0 空闲/100 占据)	9.4
PGM/YAML	nav2 地图的存盘格式 (图像+元信息)	8.8、9.4
Y-flip	OccupancyGrid 与 PGM 行序相反需翻转	9.4、10.3
costmap (代价地图)	障碍膨胀后的"危险度"格子图	12.2
AMCL	自适应蒙特卡洛 (粒子滤波) 定位	12.3、14.3
粒子滤波	用一群加权假设逼近位姿分布	14.3
似然场	预计算距离场, 用于给激光匹配打分	14.3

术语	一句话解释	详见
DWB	Nav2 的局部控制器（速度采样+评价器打分）	12.3
inflation_radius	代价地图的障碍膨胀半径	12.8
NavigateToPose	Nav2 对外的导航 action 接口	13.3
里程计漂移	轮速积分误差的累积	3.3、14.2
map→odom	定位模块发布的漂移修正量	6.3、14.3
CP (Checkpoint)	每节课的自动验收脚本	第 11/14 步

结语

回顾全程，这个项目的搭建路径可以浓缩成一句话：

先定接口（第 6 步的契约），再从数据流上游往下游一环一环搭（仿真 → 感知 → 建图 → 定位 → 规划 → 任务），每环搭完立即验证，每课结束留下自动验收。

照这个方法，你可以把本项目替换成任何机器人、任何环境—— 换个 URDF、换个世界文件、调几个参数，骨架完全复用。这正是 ROS 2 组件化设计的价值所在。

祝搭建顺利。遇到问题先翻第 15 步的速查表，再对照 git 标签里的 成品逐文件 diff，绝大多数问题都能自己定位。